

Mémoire d'ingénieur

Industrialisation de la chaîne de valeur d'une ESN

David Guichard

Année 2025–2026

Stage de fin d'études réalisé dans l'entreprise

Oxyl

en vue de l'obtention du diplôme d'ingénieur de TELECOM Nancy

Maître de stage : Nom de l'encadrant

Encadrant universitaire : Nom du tuteur académique

Déclaration sur l'honneur de non-plagiat

Je soussigné(e),

Nom, prénom : Guichard, David

Élève-ingénieur(e) régulièrement inscrit(e) en 3^e année à TELECOM Nancy

Numéro de carte de l'étudiant(e) : n°étudiant

Année universitaire : 2025–2026

Auteur(e) du document, mémoire, rapport ou code informatique intitulé :

**Industrialisation de la chaîne de valeur d'une ESN : développement
du capital humain et innovation technologique**

Par la présente, je déclare m'être informé(e) sur les différentes formes de plagiat existantes et sur les techniques et normes de citation et référence.

Je déclare en outre que le travail rendu est un travail original, issu de ma réflexion personnelle, et qu'il a été rédigé entièrement par mes soins. J'affirme n'avoir ni contrefait, ni falsifié, ni copié tout ou partie de l'œuvre d'autrui, en particulier texte ou code informatique, dans le but de me l'accaparer.

Je certifie donc que toutes formulations, idées, recherches, raisonnements, analyses, programmes, schémas ou autre créations, figurant dans le document et empruntés à un tiers, sont clairement signalés comme tels, selon les usages en vigueur.

Je suis conscient(e) que le fait de ne pas citer une source ou de ne pas la citer clairement et complètement est constitutif de plagiat, que le plagiat est considéré comme une faute grave au sein de l'Université, et qu'en cas de manquement aux règles en la matière, j'encourrais des poursuites non seulement devant la commission de discipline de l'établissement mais également devant les tribunaux de la République Française.

Fait à Nancy, le 26 août 2026

Signature :



Mémoire d'ingénieur

Industrialisation de la chaîne de valeur d'une ESN

David Guichard

Année 2025–2026

Stage de fin d'études réalisé dans l'entreprise Oxyl
en vue de l'obtention du diplôme d'ingénieur de TELECOM Nancy

David Guichard
numéro, rue
code postal, VILLE
téléphone
david.guichard@telecomnancy.eu

TELECOM Nancy
193 avenue Paul Muller,
CS 90172, VILLERS-LÈS-NANCY
+33 (0)3 83 68 26 00
contact@telecomnancy.eu

Oxyl
numéro, rue
code postal, VILLE
téléphone



Maître de stage : Nom de l'encadrant

Encadrant universitaire : Nom du tuteur académique

Remerciements

*“Night gathers, and now my watch begins.
It shall not end until my death.*

*I shall take no wife, hold no lands, father no children.
I shall wear no crowns and win no glory.
I shall live and die at my post.*

*I am the sword in the darkness.
I am the watcher on the walls.
I am the shield that guards the realms of men.*

*I pledge my life and honor to the Night’s Watch,
for this night and all the nights to come.”*

– The Night’s Watch oath

Avant-propos

Ce Projet de Fin d'Études chez Oxyl s'inscrit au carrefour de mon parcours d'ingénieur et d'une vision professionnelle mûrement réfléchie.

Dès le choix de ma spécialisation à TELECOM Nancy, j'ai délibérément privilégié la filière *Logiciel Embarqué*. Ce choix stratégique répondait à une volonté de construire un socle de compétences bas-niveau et architectural solide, indispensable pour appréhender les systèmes complexes avec une réelle rigueur d'ingénierie.

Cette passerelle entre socle logiciel et intelligence artificielle s'est dessinée dès mon stage de deuxième année au laboratoire CRAN, à travers la conception d'une architecture cognitive en *Deep Learning* pour un bras robotique en cobotique industrielle. Elle s'est ensuite consolidée lors de mon semestre d'échange académique à Madrid, dédié à l'approfondissement théorique et pratique du *Machine Learning*. Mon intégration chez Oxyl est venue parachever cette démarche, en mettant à profit cette double culture pour concevoir une usine logicielle optimisée par l'automatisation agentique.

Ce mémoire marque ainsi une étape charnière : à l'avenir, j'envisage de poursuivre dans l'automatisation des processus d'entreprise par les systèmes agentiques. Une ambition guidée par la conviction que l'efficacité de l'IA appliquée dépend avant tout de la robustesse du socle en développement logiciel sur lequel elle s'appuie.

David GUICHARD
Le Plessis-Robinson, le 18 août 2026

Table des matières

Remerciements	v
Avant-propos	vii
Table des matières	ix
1 Introduction Générale	1
2 Vers une industrialisation de la chaîne de valeur (Le cadre)	5
2.1 L'ESN comme « usine » de talent et de code : Présentation d'Oxyl et du cycle de vie du consultant	5
2.1.1 Présentation du Groupe Oxyl et positionnement stratégique	5
2.1.2 Organisation structurelle et modèle d'entreprise libérée	6
2.1.3 L'« usine de talents » : De la sélection à l'excellence certifiée	7
2.1.4 Le cycle de vie du collaborateur et la dynamique d'apprentissage continu	8
2.2 Problématique double : Analyse de la dépendance technologique (vendor lock-in) et de la fragilité du socle de compétences	9
2.2.1 La fragilité du socle de compétences : du développeur de surface à l'auditeur critique	9
2.2.2 Le risque d'enfermement technologique (<i>vendor lock-in</i>) : l'impératif d'une usine logicielle agnostique	10
2.2.3 La synergie dialectique : redéfinir la valeur de l'ingénieur à l'ère agentique	10
2.3 La stratégie d'ingénierie : Justification de la synergie entre formation continue et automatisation agentique	11
2.3.1 Anticipation du marché : les projets internes comme banc d'essai industriel	11
2.3.2 La transmission du savoir-faire : de l'expertise humaine aux garde-fous de l'usine	11
2.3.3 Agnosticité et réassurance client : la garantie d'une ingénierie mature . .	12
3 La standardisation du capital humain (Volet 1 - Le Quiz)	13
3.1 Analyse des besoins en montée en compétences : Le rôle du socle Java/JEE dans la qualité logicielle	13

3.1.1	Évolution du marché et impératif de certification industrielle	13
3.1.2	Le socle Java et la programmation orientée objet comme vecteurs de qualité	14
3.1.3	Limites de l'outillage existant et identification des besoins	14
3.1.4	Cahier des charges fonctionnel et innovations de la plateforme	14
3.1.5	Objectifs d'ingénierie et exigences implicites	15
3.2	Ingénierie de la plateforme : Conception d'un système de contrôle qualité des connaissances	16
3.2.1	Modélisation du domaine et structuration arborescente des connaissances	16
3.2.2	Architecture logicielle backend et services RESTful	17
3.2.3	Gouvernance des accès, Sécurité et Interopérabilité Front/Back	19
3.2.4	Conception de l'interface utilisateur et expérience apprenant (Angular) . .	20
3.2.5	Dynamique d'ingénierie collaborative, gestion des risques et retours d'expérience	21
3.3	Choix techniques comme choix de maturité : Justification de l'architecture (Spring Core/Angular) pour une maîtrise approfondie des fondamentaux	22
3.3.1	Démystification des couches applicatives : Le choix de Spring Core et de la configuration explicite	22
3.3.2	Adoption d'Angular : Rigueur structurelle et opportunité de montée en compétences	22
3.3.3	Monolithe modulaire vs Microservices : Pragmatisme et focalisation sur le prototype	23
3.3.4	Écosystème outillé et composants clés du socle technique	23
4	La standardisation du capital technique (Volet 2 - L'usine IA)	25
4.1	État de l'art : L'industrialisation du cycle de vie logiciel (SDLC) et les agents IA (SWE-bench)	25
4.1.1	L'évolution du SDLC et la primauté de la génération de code	25
4.1.2	Évaluation agentique par SWE-bench et assistants en ligne de commande	25
4.1.3	Analyse comparative des solutions agentiques du marché	26
4.2	Théorie de l'abstraction : Étude des patrons de conception pour s'affranchir du lock-in	26
4.2.1	Dilemme d'ingénierie et arbitrage des patrons de conception	27
4.2.2	Modélisation de la couche d'abstraction des moteurs d'IA	27
4.2.3	Défis d'abstraction et contraintes d'exécution conteneurisée	28
4.3	Analyse des contraintes : Sécurité, gestion des coûts de tokens et reproductibilité des résultats	29
4.3.1	Maîtrise des coûts et des temps d'exécution : Le double verrou de réessai .	29
4.3.2	Gestion du flux de sortie et compression contextuelle : Le rôle du Manifest	30

4.3.3	Sécurité, étanchéité des conteneurs et assainissement des secrets	30
4.3.4	Gouvernance déterministe : L'arsenal des garde-fous mécaniques	31
4.3.5	Reproductibilité, métriques et perspective du patron Observer	32
5	La mise en œuvre de l'infrastructure agnostique	33
5.1	Architecture du moteur agnostique : Le découplage des couches Git et des couches LLM	33
5.1.1	Conception modulaire et contrats d'interface : BacklogProvider et Code-BaseProvider	33
5.1.2	Typage nominal étiqueté (<i>Branded Types</i>) et résolution dynamique des instances	34
5.1.3	Cycle opérationnel : Isolation par <i>Git Worktrees</i> et suivi des tâches	35
5.1.4	Exécution du pipeline agentique et publication déterministe de la MR . . .	35
5.1.5	Schéma d'architecture globale du moteur agnostique	36
5.2	Stratégies de normalisation : Uniformisation des API hétérogènes (GitLab/GitHub/-Gitea/LLMs)	36
5.2.1	Modèle pivot canonique : Unification des entités métier (<i>Data Mapper</i>) . .	37
5.2.2	Abstraction et interchangeabilité des moteurs agentiques (<i>AgentProvider</i>)	38
5.2.3	Protocole pivot d'échange documentaire : Les artefacts Markdown (<code>.task/*.md</code>)	39
5.3	Validation de la chaîne de valeur : Faisabilité de bout-en-bout et stratégie d'arbitrage des LLMs	39
5.3.1	Validation opérationnelle et retours d'expérience sur projets réels	39
5.3.2	Repositionnement du rôle de l'ingénieur : Le paradigme <i>Human-in-the-Loop</i>	40
5.3.3	État du déploiement des moteurs LLM et impératif d'agnosticisme	40
5.3.4	Matrice et critères d'arbitrage pour le benchmarking futur	41
6	Analyse critique et Impact systémique	43
6.1	Synergie entre les deux volets : Comment le socle de compétences facilite la maintenance et l'évolution de l'usine logicielle	43
6.1.1	La boucle de rétroaction bidirectionnelle et le déplacement de la charge cognitive	43
6.1.2	L'exigence accrue d'un socle théorique face aux heuristiques de l'IA . . .	44
6.1.3	Transversalité des paradigmes : du socle Java OCP à l'architecture TypeScript de l'usine	44
6.2	Impact économique et technologique : ROI, réduction du vendor lock-in, gains de productivité et attractivité client	45
6.2.1	Modélisation du retour sur investissement (ROI) et gains de productivité .	45
6.2.2	Optimisation financière par l'arbitrage dynamique et l'anti-lock-in	46

6.2.3	Valorisation du capital humain : Réduction du coût de non-qualité et levier commercial	47
6.2.4	Attractivité client et renouvellement du modèle de service de l'ESN	48
6.3	Recul critique : Limites de l'IA agentique, enjeux éthiques et de confidentialité . .	48
6.3.1	Limites techniques et opérationnelles : heuristiques, dérives de raisonnement et atomicité	48
6.3.2	Sécurité, confidentialité et gouvernance des données du code source . . .	49
6.3.3	Impacts humains, posture de l'ingénieur et transmission des compétences	50
7	Conclusion Générale	51
	Bibliographie / Webographie	55
	Liste des illustrations	57
	Liste des tableaux	59
	Listings	61
	Annexes	64
A	Attestations des certifications obtenues	65
B	Diagrammes de classe / Schémas d'architecture	67
C	Exemples de traces d'exécution de l'agent IA	69
	Résumé	71
	Abstract	71

1 Introduction Générale

Contexte sectoriel : La métamorphose du modèle des ESN

Le secteur des Entreprises de Services du Numérique (ESN) traverse une phase de mutation structurelle sans précédent, impulsée par l'émergence et la maturation rapide de l'intelligence artificielle générative et des architectures agentiques. Historiquement, le modèle économique de l'ingénierie logicielle de prestation reposait en grande partie sur l'assistance technique en régie, valorisée au temps passé. Aujourd'hui, la dynamique du marché impose une transition accélérée vers des modèles au forfait et des engagements au résultat. Dans ce paradigme, les cycles de livraison sont drastiquement raccourcis, la prévisibilité des charges devient un enjeu critique de rentabilité, et la moindre dérive dans l'estimation des délais fait peser un risque direct sur la pérennité de la relation client.

Parallèlement, l'automatisation progressive des phases élémentaires du cycle de développement logiciel (*Software Development Life Cycle* – SDLC) bouleverse la gestion des talents. Les tâches jadis confiées aux développeurs juniors sont de plus en plus prises en charge par des agents d'assistance au code, fermant l'accès aux missions traditionnelles d'intégration pour les profils en début de carrière. En outre, le marché fait face à une dichotomie marquée au sein de la clientèle : tandis qu'une fraction exige une vélocité accrue portée par l'IA, une part significative d'entreprises – notamment dans les secteurs réglementés ou stratégiques – interdit formellement l'utilisation d'outils d'IA tiers pour prévenir tout risque de fuite de données sensibles ou d'atteinte à la propriété intellectuelle. Pour une ESN, l'équation impose ainsi de maintenir une rigueur méthodologique humaine irréprochable tout en intégrant l'automatisation logicielle de pointe.

L'écosystème Oxyl et sa stratégie d'innovation

Dans cet environnement concurrentiel tendu, Oxyl a fait le choix stratégique de se positionner très tôt comme une entreprise pionnière sur l'intégration de l'intelligence artificielle dans ses processus d'ingénierie logicielle. L'ambition d'Oxyl repose sur une vision duale : acquérir une expertise de premier plan sur les systèmes agentiques pour concevoir des produits à haute valeur ajoutée, tout en décuplant la performance opérationnelle de ses propres consultants en mission.

Cette posture d'innovation s'accompagne d'une politique de formation et de gestion du capital humain ciblée. Afin de maintenir une dynamique perpétuelle de veille technologique et de renouvellement intellectuel, Oxyl privilégie le recrutement par la continuité de stage de fin d'études. L'accueil d'élèves-ingénieurs permet d'éprouver en temps réel les dernières avancées académiques et techniques face aux exigences concrètes de la production industrielle.

Le double défi d'ingénierie : Standardisation du capital humain et du capital technique

Pour concrétiser cette vision industrielle, le projet d'ingénierie mené durant ce stage s'articule autour de deux axes fondamentaux et complémentaires :

1. **La standardisation et la valorisation du capital humain (Le volet compétences) :**
Face aux exigences accrues des clients, l'excellence technique des collaborateurs doit être démontrable et certifiée. L'obtention de certifications reconnues par l'industrie – à l'instar des titres Oracle (*Oracle Certified Associate* – OCA, *Oracle Certified Professional* – OCP) pour l'écosystème Java/JEE, ou de l'accréditation *AWS Certified Cloud Practitioner* – constitue un puissant gage de crédibilité. Pour soutenir cet impératif, une plateforme web sur-mesure a été conçue en Spring Core et Angular. Pensée à la fois comme un *moyen* (un projet grandeur nature formant aux standards d'architecture) et comme un *but* (un outil d'entraînement, de passage d'examens blancs et de suivi statistique pour les encadrants), cette plateforme structure la montée en compétences et la gouvernance pédagogique interne.
2. **La standardisation et l'indépendance du capital technique (Le volet Usine Logicielle) :**
Pour automatiser les tâches répétitives du cycle de vie logiciel sans subir les aléas des solutions propriétaires, Oxyl a initié le développement d'une « Usine Logicielle » agentique en TypeScript, régie par des spécifications formelles en Markdown (prompts, descriptions d'incidents et règles d'architecture logicielle propre). Face à la volatilité des coûts de facturation des jetons (*tokens*), aux impératifs de confidentialité et à la diversité des infrastructures clientes, un enjeu architectural majeur est apparu : garantir l'agnosticité complète du moteur. Il s'agissait de découpler l'infrastructure d'exécution vis-à-vis des fournisseurs de modèles de langage (OpenAI, Anthropic Claude, Google, modèles locaux) et des gestionnaires de dépôts et de tickets (GitHub, GitLab, Gitea, Jira, Bitbucket).

Problématique et objectifs du Projet de Fin d'Études

L'intersection entre l'exigence d'un socle d'ingénierie rigoureux et l'impératif d'industrialisation automatisée soulève la problématique centrale de ce mémoire :

Dans un marché des ESN contraint par le passage au forfait et la transformation des métiers du développement par l'IA, comment concilier la standardisation du capital humain (gage de qualité et de maîtrise des fondamentaux) et l'industrialisation d'une usine logicielle agentique agnostique pour pérenniser la rentabilité et l'indépendance technologique de l'entreprise ?

Cadre méthodologique et dynamique d'équipe

La réalisation de ces travaux s'est inscrite dans deux contextes méthodologiques complémentaires, illustrant la diversité des modes d'organisation en ingénierie logicielle :

- **Le projet de la plateforme d'évaluation (Équipe de 4 stagiaires)** : Mené *from scratch*, le projet a débuté selon des cycles Scrum hebdomadaires rythmés par des revues de code individuelles exigeantes, conditionnant la validation de chaque incrément. La phase finale sur le développement front-end s'est ensuite déroulée en semi-autonomie sous forme de flux Kanban (points quotidiens, découpage atomique des tâches). Cette configuration a constitué un exercice pratique immersif sur la gestion de la dette technique, l'arbitrage entre vélocité et maintenabilité, et le respect d'échéances strictes.
- **Le projet Usine Logicielle (Organisation Agile d'entreprise)** : Intégré au sein d'une équipe structurée par un *Scrum Master* avec un backlog rigoureusement priorisé, le travail a suivi des rituels réguliers (*dailies*, synchronisations hebdomadaires). La culture de qualité logicielle y a été renforcée par une pratique intensive de la revue de code collective (une à deux sessions quotidiennes), permettant d'aligner en continu les décisions d'architecture et les principes de *Clean Code*.

Synthèse des contributions personnelles

Bien que s'inscrivant dans une démarche collaborative, mes contributions d'ingénieur ont plus particulièrement porté sur :

- Le développement *full-stack* de la plateforme d'évaluation des compétences, notamment la modélisation des entités, les API de gestion des référentiels pédagogiques et l'implémentation des parcours de tests.
- La conception et l'implémentation de la couche d'abstraction logicielle au sein de l'Usine Logicielle, spécifiquement l'interopérabilité des connecteurs de gestionnaires de code source (*CodebaseProviders* : GitHub, GitLab, Gitea, Bitbucket) et des systèmes de suivi de tickets (*BacklogProviders* : Jira, GitLab/GitHub Issues).
- L'évaluation et la consolidation des patrons de conception (*Design Patterns*) assurant l'extensibilité et la résilience du moteur face à l'hétérogénéité des écosystèmes clients.

Organisation du mémoire

Le présent mémoire est structuré en cinq chapitres qui retracent la démarche d'ingénierie adoptée :

- Le **Chapitre 2** détaille le cadre de l'étude, en analysant l'écosystème d'Oxyl, le cycle de vie du consultant, les risques de dépendance technologique (*vendor lock-in*) et la justification stratégique du couplage entre formation et automatisation.
- Le **Chapitre 3** présente le premier volet consacré à la standardisation du capital humain, en détaillant l'architecture de la plateforme d'évaluation (Spring Core / Angular) et son rôle dans la maîtrise des fondamentaux d'ingénierie.
- Le **Chapitre 4** aborde le second volet dédié au capital technique, à travers l'état de l'art sur l'industrialisation du SDLC par les agents autonomes, la théorie de l'abstraction logicielle et la gestion des contraintes opérationnelles (coûts et sécurité).
- Le **Chapitre 5** décrit la mise en œuvre de l'infrastructure agnostique, détaillant les patrons de conception retenus, l'uniformisation des interfaces de programmation et les résultats expérimentaux obtenus.
- Le **Chapitre 6** propose une analyse critique et systémique du projet, mesurant les retours sur investissement, la réduction des dépendances propriétaires et les perspectives éthiques

de l'IA appliquée au génie logiciel.

- Enfin, la **Conclusion Générale** synthétisera les apports du projet et ouvrira sur l'évolution prospective du métier d'ingénieur logiciel.

2 Vers une industrialisation de la chaîne de valeur (Le cadre)

2.1 L'ESN comme « usine » de talent et de code : Présentation d'Oxyl et du cycle de vie du consultant

Dans une économie numérique fortement concurrentielle et marquée par l'accélération technologique, le modèle traditionnel des Entreprises de Services du Numérique (ESN) est confronté à un impératif de transformation. Pour répondre aux exigences de rentabilité, de vitesse et d'excellence technique imposées par les donneurs d'ordre, Oxyl a développé un modèle singulier assimilant l'entreprise à une véritable double « usine » : d'une part une « usine de talents », chargée de recruter, former et standardiser les compétences d'ingénieurs à haut potentiel, et d'autre part une « usine de code », outillée pour industrialiser la production logicielle grâce aux architectures modernes et à l'automatisation agentique.

2.1.1 Présentation du Groupe Oxyl et positionnement stratégique

Fondée initialement en 1995 par Fabrice REBY et des élèves-ingénieurs de l'École Centrale de Lille sous le nom d'*AlphaCSP*, l'entreprise s'est d'abord distinguée par son rôle de pionnière dans l'adoption industrielle des technologies Java/J2EE en France, en nouant des partenariats technologiques de premier plan avec des acteurs historiques tels qu'IBM, BEA Systems ou Borland. Précurseur également dans la diffusion des démarches agiles (notamment Scrum et eXtreme Programming) dès le début des années 2000, la société a fait évoluer sa structure et son identité pour devenir le **Groupe Oxyl** (Digital Makers).

Aujourd'hui implanté à Arcueil en région parisienne, le groupe rassemble environ 200 collaborateurs et a réalisé en 2024 un chiffre d'affaires consolidé de près de 20 millions d'euros. Oxyl se positionne comme un cabinet de conseil en technologies et une ESN de niche à forte valeur ajoutée, axée sur la maîtrise approfondie du cycle de vie logiciel (*Software Development Life Cycle* – SDLC) : de la conception architecturale au déploiement continu en environnement Cloud, en passant par le développement et le maintien en condition opérationnelle (MCO).

Sur le plan technique, Oxyl concentre son cœur de métier autour de deux écosystèmes majeurs et de leurs extensions industrielles :

- **L'écosystème Java / JEE & Spring** : architectures distribuées, microservices, optimisation des performances transactionnelles, mapping objet-relationnel et sécurité applicative ;
- **L'écosystème JavaScript / TypeScript** : développement d'applications web réactives modernes (Angular, React, Vue.js, Node.js) ;
- **L'ingénierie Cloud & DevOps** : conteneurisation (Docker, Kubernetes), infrastructures as

Code (Terraform, Ansible), pipelines CI/CD et services managés chez les principaux fournisseurs de Cloud public (Amazon Web Services, Microsoft Azure, Google Cloud Platform);

- **L’intelligence artificielle et l’automatisation logicielle** : conception d’architectures agenticues, intégration de modèles de langage (LLM) dans les flux de développement et R&D sur la résolution autonome de tickets.

Face aux grandes entreprises généralistes du secteur (telles que Capgemini, Sopra Steria ou Atos) et aux ESN de taille intermédiaire (comme Zenika ou Ippon Technologies), Oxyl fonde son avantage concurrentiel sur une triple différenciation :

1. **Une hyper-spécialisation technique** : un positionnement volontairement ciblé sur un socle de technologies maîtrisées d’excellence, évitant la dispersion technologique ;
2. **Une excellence démontrable et certifiée** : la garantie pour les clients d’intégrer des ingénieurs dont le niveau théorique et pratique est validé et attesté par des certifications officielles dès le démarrage des missions ;
3. **Une agilité et une proximité opérationnelle** : la réactivité d’une structure à taille humaine alliée à un ancrage fort en région parisienne.

Cette expertise s’exprime auprès d’un portefeuille de clients diversifiés, comprenant des grands comptes du secteur bancassurance et de la finance (Allianz Trade, BPCE - Casden, Société Générale, MyMoneyBank), des leaders de l’industrie et des services (Rexel, Lafarge, Eutelsat, JCDecaux, Vinci), ainsi que des éditeurs de logiciels et des startups technologiques (Finance Active, Wedia, PrimaSolutions, Hologarde, Wanaka).

2.1.2 Organisation structurelle et modèle d’entreprise libérée

Pour préserver l’agilité organisationnelle et stimuler l’engagement de ses collaborateurs, Oxyl s’est structurée autour des principes de l’« entreprise libérée ». Ce paradigme managérial repose sur l’autonomie, la subsidiarité, la réduction des échelons hiérarchiques intermédiaires et la responsabilisation directe des ingénieurs dans les décisions techniques et la vie de l’entreprise.

La culture d’Oxyl s’articule autour de trois valeurs cardinales :

- **La Passion** : inspirée par la maxime de Confucius (« *Choisissez un travail que vous aimez et vous n’aurez pas à travailler un seul jour de votre vie* »), l’entreprise s’attache à recruter des profils sincèrement passionnés par l’ingénierie logicielle et l’innovation ;
- **L’Excellence** : sur un marché exigeant, la qualité de prestation repose sur la rigueur technique, le respect des standards de conception (*Clean Code*, patrons de conception) et une veille technologique continue ;
- **Le Partage** : l’entraide mutuelle entre pairs, la transmission du savoir intergénérationnel et le partage de la valeur créée constituent le socle de la cohésion collective.

Sur le plan juridique et opérationnel, le groupe s’appuie sur une structure bicéphale synergique :

- **Sylex (L’entité transverse de support)** : structure regroupant les fonctions centrales de l’entreprise (direction générale, administration des ventes, ressources humaines, gestion financière et comptable, prospection commerciale et logistique événementielle) ;
- **Les entités Oxyl par promotion (Oxyl, Oxyl 16 à 23, Oxyl+)** : regroupement des consultants par promotion d’intégration, garantissant le maintien de cellules à échelle humaine afin de pérenniser les liens de confraternité et le sentiment d’appartenance.

La gouvernance opérationnelle s’organise autour d’une équipe de direction resserrée : M. Fabrice REBY (fondateur, apportant sa vision stratégique et son portefeuille commercial), Mme Selena HIND-WOODWARD (responsable du pôle administratif, RH, relations écoles et suivi des carrières), M. Georges COUDRAY (responsable du pôle commercial et du staffing des consultants), ainsi que

M. Maxime BAUER (directeur technique, assurant l'encadrement technologique, la supervision des formations et le pilotage des projets internes de R&D).

2.1.3 L'« usine de talents » : De la sélection à l'excellence certifiée

Le concept d'« usine de talents » résume la démarche hautement industrialisée et reproductible mise en place par Oxyl pour transformer des étudiants ingénieurs en consultants immédiatement opérationnels et armés face aux exigences les plus pointues de l'industrie.

Un processus de sélection ciblé et rigoureux

Le recrutement constitue le premier maillon de cette chaîne de valeur. Présente activement dans les grandes écoles d'ingénieurs à travers plus de 40 conférences techniques annuelles, Oxyl attire chaque année plus de 2 000 candidatures. Le processus de sélection suit un entonnoir strict à plusieurs étapes :

1. **Test technique initial en ligne (1 heure)** : évaluation des compétences fondamentales en algorithmique et en programmation (taux de sélection : environ 2 candidats retenus sur 5);
2. **Entretien de qualification culturelle** : échange téléphonique mené par un collaborateur pour évaluer la passion du candidat, sa curiosité et son adéquation aux valeurs humaines de l'entreprise (taux d'élimination : environ 30 %);
3. **Test pratique de développement (1 heure) & Débriefing technique** : réalisation d'un développement concret sur la base d'un mini cahier des charges, suivi d'une soutenance technique avec un référent pour analyser la démarche d'ingénierie et le raisonnement adopté;
4. **Entretien de synthèse managérial** : validation finale de l'embauche en stage de fin d'études avec les managers du groupe.

Au terme de ce filtre, environ 40 stagiaires sont intégrés chaque année, répartis en promotions régulières selon leur date d'arrivée. J'ai pour ma part intégré la promotion d'**avril 2026**. Afin de faciliter l'accueil et l'installation en région parisienne des élèves-ingénieurs issus d'écoles régionales (à l'instar de TELECOM Nancy), Oxyl propose un parc d'appartements en colocation situés à proximité du siège, accélérant ainsi la cohésion d'équipe dès les premiers jours.

Le parcours de formation initiale standardisé

Durant les trois premiers mois du stage, l'ensemble des recrues suit un programme de formation intensive conçu pour homogénéiser le niveau technique et bâtir des réflexes d'ingénierie irréprochables. Ce parcours s'articule autour de deux composantes majeures :

1. La validation par les certifications officielles : Loin d'un simple apprentissage déclaratif, Oxyl impose la validation de certifications industrielles reconnues internationalement, faisant office de tiers de confiance indiscutable pour les clients :

- *Oracle Certified Associate (OCA) Java SE 8 Programmer I* : maîtrise rigoureuse des mécanismes fondamentaux du langage (cycle de vie, typage, modélisation objet, gestion des exceptions);
- *Oracle Certified Professional (OCP) Java SE 21 Developer* : assimilation poussée des paradigmes avancés (programmation fonctionnelle, Streams, concurrence, I/O non bloquantes, modules JPMS);

- *AWS Certified Cloud Practitioner* : acquisition des bases de l'architecture Cloud, des modèles de sécurité partagée et des services managés d'Amazon Web Services.

Cette préparation s'appuie sur la plateforme interne **MyNeuroFactory** (entraînement quotidien de 1 heure le matin et examens blancs surveillés chaque vendredi après-midi).

2. Le projet fil rouge « NewroFactory » (L'apprentissage par l'épreuve) : En parallèle de l'étude théorique, chaque stagiaire doit redévelopper en autonomie une version complète de l'application de formation. Découpé en 8 sprints itératifs, le projet force l'ingénieur à concevoir l'application d'abord en code bas niveau natif (CLI en Java pur, Servlets, JDBC), pour lui faire ressentir concrètement les limitations architecturales avant d'introduire successivement les couches d'abstraction industrielles (pools HikariCP, inversion de contrôle avec Spring Core, MVC, programmation orientée aspect – AOP, ORM Hibernate/JPA, Spring Security, puis exposition en API RESTful et interface front-end). Chaque sprint est validé lors d'une revue de code (*code review*) collective et intransigeante menée par la direction technique, où le respect des principes de *Clean Code* (SOLID, DRY, KISS, YAGNI) est systématiquement audité.

2.1.4 Le cycle de vie du collaborateur et la dynamique d'apprentissage continu

À l'issue de cette phase d'accélération initiale de trois mois, le modèle d'Oxyl oriente les consultants vers deux trajectoires opérationnelles complémentaires selon les besoins stratégiques :

- **Le parcours « Padawan » en immersion client :** l'ingénieur est introduit en régie chez un client de l'ESN, sous le tutorat bienveillant d'un consultant senior d'Oxyl (le « Jedi »). Ce dispositif, proposé sans refacturation additionnelle durant le stage, garantit une intégration sereine dans des écosystèmes industriels complexes et sécurise l'embauche en CDI à l'issue du cursus.
- **Le parcours R&D et Projets Internes :** l'ingénieur rejoint l'équipe interne de développement d'Oxyl, dédiée à la conception de solutions innovantes et d'outils d'assistance logicielle (tels que l'extension *OxaPocket* ou l'*Usine Logicielle* agentique). Ce pôle constitue un véritable banc d'essai pour tester les ruptures technologiques et sert de vitrine de savoir-faire pour l'entreprise.

Ce cycle de vie professionnel est entretenu tout au long de la carrière du collaborateur par des dispositifs pérennes d'intelligence collective :

- **Les TechNights mensuelles :** conférences internes organisées le jeudi soir au siège, où les consultants en mission présentent des retours d'expérience (REX) ou explorent de nouveaux paradigmes (ex. Rust, Kotlin, Claude Code, architectures agentiques);
- **Les Café Tech :** ateliers réguliers de veille technique en binômes;
- **Le portail de veille collective :** base de connaissances collaborative permettant de solliciter les experts internes et de capitaliser sur les solutions éprouvées en mission.

En instaurant cette rigueur méthodologique dès l'intégration, Oxyl standardise son capital humain. Cette première étape est la condition indispensable pour aborder le second volet de sa chaîne de valeur : l'industrialisation du capital technique et la mise en place d'une « usine de code » automatisée par l'intelligence artificielle.

2.2 Problématique double : Analyse de la dépendance technologique (vendor lock-in) et de la fragilité du socle de compétences

L'ambition d'Oxyl d'articuler son modèle autour d'une double « usine » (de talents et de code) s'inscrit au cœur d'une tension structurelle du secteur informatique. D'un côté, le capital humain fait face à une érosion de la maîtrise des couches basses de l'ingénierie logicielle ; de l'autre, le capital technique est exposé à un risque critique d'enfermement propriétaire face à l'avènement des modèles d'intelligence artificielle générative. La pérennité et la compétitivité de l'entreprise reposent ainsi sur la résolution conjointe de cette double problématique.

2.2.1 La fragilité du socle de compétences : du développeur de surface à l'auditeur critique

Un paradoxe frappant caractérise la formation contemporaine des jeunes diplômés en informatique : alors qu'ils sont initiés très tôt à des frameworks modernes de haut niveau, une proportion significative d'élèves-ingénieurs intègre le monde professionnel avec une compréhension purement superficielle et strictement utilitaire des concepts. Trop souvent, l'apprentissage privilégie l'usage magique des abstractions (annotations, bibliothèques prêtes à l'emploi) au détriment d'une compréhension intime de ce qui s'exécute « sous le capot » (gestion de la mémoire JVM, cycle de vie des objets, mécanismes transactionnels, concurrence non-bloquante ou requêtage SQL natif).

Cette dérive cognitive est aujourd'hui amplifiée par l'omniprésence des assistants de programmation génératifs (tels que GitHub Copilot ou ChatGPT). En générant instantanément des blocs de code fonctionnels en apparence, ces outils installent une illusion de compétence qui masque les lacunes structurelles. L'ingénieur devient alors un simple assembleur de composants qu'il ne maîtrise pas, s'exposant à des risques majeurs :

- **L'introduction de failles invisibles et insidieuses** : des anomalies architecturales, des vulnérabilités de sécurité ou des fuites de ressources qui échappent aux tests unitaires de surface et ne se manifestent qu'en production sous forte charge ou lors d'attaques ciblées ;
- **L'incapacité à maintenir et faire évoluer des systèmes patrimoniaux (*legacy*)** : chez les clients grands comptes, les consultants d'Oxyl sont fréquemment confrontés à des architectures hétérogènes, parfois anciennes ou atypiques. L'adaptation rapide à ces contextes exige une solide culture des concepts fondamentaux pour comprendre rapidement les contraintes sous-jacentes et intervenir avec pertinence ;
- **L'incapacité à auditer le code produit par des tiers ou des agents** : un développeur ne possédant qu'une compréhension moyenne ne peut identifier les erreurs subtiles d'implémentation.

Face à ce constat, l'obligation d'un apprentissage *from scratch* (de la programmation système et JDBC brut jusqu'aux architectures d'entreprise) s'impose comme une nécessité absolue pour forger des ingénieurs capables d'appréhender les moindres détails techniques et d'exercer un regard critique intransigeant.

2.2.2 Le risque d'enfermement technologique (*vendor lock-in*) : l'impératif d'une usine logicielle agnostique

Parallèlement à la qualification du capital humain, l'automatisation de la chaîne de production logicielle par l'intelligence artificielle soulève un défi d'indépendance technologique sans précédent. L'intégration précipitée d'outils d'IA fermés au sein du cycle de développement (SDLC) expose les organisations à un triple risque de verrouillage :

1. **L'hétérogénéité des environnements clients** : Oxyl intervient auprès d'une multiplicité de donneurs d'ordre, chacun imposant ses propres contraintes de sécurité, ses choix d'infrastructure (Cloud souverain, environnements *on-premise*, conteneurisation privée) et ses stacks logicielles. Une usine de code interne ne peut être viable que si elle est capable de s'adapter dynamiquement à ces variables sans exiger une refonte de son architecture ;
2. **La volatilité économique et l'instabilité des API** : le modèle économique des fournisseurs de LLMs (OpenAI, Anthropic, Google, etc.) repose sur une tarification au jeton (*token*) sujette à de fortes variations. Coupler rigidement une usine logicielle à une interface propriétaire exposerait l'entreprise à une dépendance financière et opérationnelle intenable ;
3. **La concurrence permanente entre modèles de langage** : l'état de l'art des modèles d'IA évolue à un rythme hebdomadaire. Un fournisseur surpassant les références du marché aujourd'hui peut être déclassé le mois suivant par un modèle concurrent ou une solution *open-source* spécialisée (ex. DeepSeek, Llama). Pour maintenir un avantage concurrentiel maximal, l'usine logicielle doit pouvoir substituer son moteur d'inférence sous-jacent de manière totalement transparente.

L'enjeu architectural consiste donc à concevoir une plateforme agentique rigoureusement agnostique, découplée des API tierces par des patrons d'abstraction formels, garantissant une flexibilité totale face aux évolutions du marché.

2.2.3 La synergie dialectique : redéfinir la valeur de l'ingénieur à l'ère agentique

Loin de constituer deux démarches isolées, la standardisation du capital humain (Volet 1) et l'industrialisation du capital technique (Volet 2) entretiennent une relation d'interdépendance fondamentale.

À mesure que les usines logicielles agentiques automatisent la génération de code standard et résolvent des tâches répétitives, la valeur intrinsèque de l'ingénieur se déplace radicalement :

Un ingénieur doté de compétences simplement moyennes n'apporte pas plus de valeur qu'une usine logicielle automatisée. À l'ère de l'intelligence artificielle, la valeur ajoutée humaine se concentre exclusivement sur les deux extrêmes du spectre d'ingénierie : l'analyse microscopique des détails d'implémentation et la vision macroscopique de l'architecture.

D'une part, au niveau **microscopique**, l'ingénieur devient un relecteur de code (*code reviewer*) d'élite. Il doit être doté d'une acuité technique suffisamment pointue pour détecter ce que l'IA a laissé passer : failles de concurrence, fuites de mémoire sournoises, inefficacités algorithmiques ou non-respect des standards d'entreprise.

D'autre part, au niveau **macroscopique**, l'ingénieur intervient comme le garant de la cohérence métier et architecturale. C'est à lui qu'incombe la responsabilité d'aligner le système sur les besoins complexes du domaine, d'arbitrer les compromis de conception globaux et de diriger les agents

autonomes. Cette capacité de discernement global, indissociable de l'expérience et d'une solide culture d'ingénierie, constitue le rempart ultime contre les dérives fonctionnelles des systèmes automatisés.

La double problématique d'Oxyl réside ainsi dans la construction d'un équilibre vertueux : forger des ingénieurs d'excellence capables de dominer intellectuellement les outils qu'ils utilisent, tout en concevant une usine logicielle résiliente et souveraine capable de démultiplier leur efficacité.

2.3 La stratégie d'ingénierie : Justification de la synergie entre formation continue et automatisation agentique

Loin de constituer deux initiatives déconnectées, la standardisation du capital humain (Volet 1) et l'industrialisation du capital technique (Volet 2) forment le cœur d'une stratégie d'ingénierie intégrée chez Oxyl. Face aux mutations profondes induites par l'intelligence artificielle générative dans l'industrie logicielle, cette stratégie répond à un impératif double : préparer l'entreprise aux futures pratiques du marché tout en garantissant un niveau de qualité irréprochable face aux exigences de ses clients grands comptes.

2.3.1 Anticipation du marché : les projets internes comme banc d'essai industriel

Le marché de la prestation de services numériques traverse une phase de transition asymétrique. D'un côté, les modèles d'intelligence artificielle agentique démontrent un potentiel de rupture pour le cycle de développement logiciel (SDLC) ; de l'autre, les grands comptes institutionnels (notamment dans la bancassurance et l'industrie) manifestent une frilosité légitime vis-à-vis de l'intégration directe de solutions d'IA au sein de leurs dépôts de code de production, freinés par des contraintes de confidentialité, de sécurité et de conformité réglementaire.

Dans ce contexte, la maîtrise de l'outillage IA constitue d'ores et déjà un facteur différenciant pour valoriser le profil des consultants en mission. Néanmoins, pour ne pas subir passivement les évolutions technologiques, Oxyl a fait le choix stratégique d'utiliser son pôle de R&D et ses projets internes comme banc d'essai pour son usine logicielle agentique. Cette démarche proactive permet à l'entreprise d'éprouver, d'optimiser et de fiabiliser ses outils d'automatisation en conditions réelles, afin d'être immédiatement opérationnelle et force de proposition dès que les organisations avant-gardistes ouvriront leurs infrastructures à ces nouveaux paradigmes.

Par ailleurs, l'IA ne se limite pas à un outil de génération : elle agit comme un accélérateur cognitif majeur au service de la formation continue des ingénieurs, facilitant l'assimilation rapide de nouvelles stacks techniques et amplifiant la dynamique d'apprentissage.

2.3.2 La transmission du savoir-faire : de l'expertise humaine aux garde-fous de l'usine

L'ordonnancement chronologique du stage — débutant par trois mois d'immersion technique intensive *from scratch* sur *NewroFactory* avant d'aborder les briques de l'usine logicielle — découle

d’une nécessité technique fondamentale : un ingénieur ne peut piloter, auditer ou contraindre une chaîne de génération automatique sur des aspects qu’il ne maîtrise pas lui-même en profondeur.

Ce que le développeur assimile durant sa formation (règles de *Clean Code*, architecture en couches, gestion fine des transactions et patrons de conception) constitue la matière première injectée dans l’usine logicielle :

- **Le cadrage comportemental par les *system prompts*** : l’expertise de l’ingénieur permet de formaliser les directives, les rôles spécialisés (*personas*) et les contraintes de conception imposés aux agents lors des phases de planification et de rédaction ;
- **L’outillage de validation déterministe** : la rigueur méthodologique acquise permet de concevoir des scripts d’évaluation stricts (compilation rigoureuse, tests unitaires et d’intégration, analyse statique via SonarQube, vérification de types) servant de portes de qualité (*quality gates*) incontournables avant toute proposition de modification.

À mesure que l’usine gagne en maturité et produit un code de plus en plus fiable, le temps consacré aux revues de code triviales diminue. Ce gain de productivité libère une charge cognitive précieuse, permettant à l’ingénieur d’élever sa posture professionnelle : il consacre désormais l’essentiel de son temps à la réflexion architecturale, à la modélisation du domaine métier et à la supervision stratégique des comportements de l’usine.

2.3.3 Agnosticité et réassurance client : la garantie d’une ingénierie mature

Face aux réticences des directions informatiques clientes, la stratégie d’Oxyl repose sur un contrat de confiance fondé sur l’excellence démontrable de ses ingénieurs. Un consultant doté d’une solide discipline technique n’acceptera jamais qu’une chaîne automatisée produise des livrables inférieurs aux standards industriels en vigueur. L’exigence du capital humain sert ainsi de caution morale et technique à la production du capital automatisé.

Enfin, le choix d’une architecture résolument agnostique — découplée des fournisseurs de modèles de langage (LLMs) et des forges logicielles (GitLab, GitHub, Bitbucket) — constitue un marqueur d’ingénierie décisif. En génie logiciel, la capacité à généraliser et abstraire un processus n’intervient qu’après la preuve de son plein fonctionnement opérationnel. Démontrer une indépendance technologique totale prouve que l’usine logicielle dépasse le stade du prototype expérimental pour atteindre celui de produit industriel robuste. Ce découplage facilite l’adoption future chez les clients en leur permettant d’intégrer l’usine directement au sein de leur propre écosystème d’outils, sans friction ni remise en cause de leurs standards établis.

Ce rapport s’articule ainsi autour de la concrétisation de cette stratégie d’ingénierie à double détente :

- Le **Chapitre 3** détaille la standardisation du capital humain à travers la conception et le développement de la plateforme de formation *NewroFactory* (Volet 1) ;
- Les **Chapitres 4 et 5** explorent l’industrialisation du capital technique, depuis l’état de l’art agentique jusqu’à la mise en œuvre de l’infrastructure logicielle agnostique (Volet 2) ;
- Enfin, le **Chapitre 6** propose une analyse critique systémique de cette synergie, mesurant ses retombées industrielles, ses gains de productivité et ses limites éthiques.

3 La standardisation du capital humain (Volet 1 - Le Quiz)

3.1 Analyse des besoins en montée en compétences : Le rôle du socle Java/JEE dans la qualité logicielle

La standardisation du capital humain au sein d'une Entreprise de Services du Numérique (ESN) repose sur une adéquation stricte entre la maîtrise des fondamentaux théoriques et les exigences opérationnelles des clients. Cette section explicite les facteurs de marché et les impératifs d'ingénierie ayant conduit à la définition du socle Java/JEE, détaille les limites de l'outillage existant et formalise le cahier des charges fonctionnel de la plateforme *NewroFactory*.

3.1.1 Évolution du marché et impératif de certification industrielle

Dans les premières phases d'expansion du secteur du numérique, la tension extrême sur le marché du recrutement conduisait les entreprises à embaucher des profils d'ingénieurs et de développeurs sans exiger de garanties formelles préalables. L'accès rapide aux projets primait sur la vérification systématique et objective des acquis techniques.

Aujourd'hui, le niveau d'exigence des donneurs d'ordre s'est considérablement accru. Les clients attendent des ESN des garanties tangibles sur la qualification de leurs consultants afin de sécuriser des projets d'envergure, soumis à de fortes contraintes de maintenabilité et de scalabilité. Pour répondre à cette exigence de réassurance, Oxyl a instauré un passage obligatoire de certifications industrielles reconnues dès la phase d'intégration des nouveaux collaborateurs :

- **Oracle Certified Associate (OCA) & Oracle Certified Professional (OCP) Java SE** : validation formelle de la maîtrise approfondie du langage Java, de sa syntaxe, de la gestion mémoire, du modèle de concurrence et des patrons de conception objet ;
- **AWS Certified Cloud Practitioner** : certification attestant de la compréhension des architectures distribuées, des services cloud fondamentaux, de la sécurité et des modèles de déploiement industriels.

L'obtention de ces certifications ne constitue pas un simple label commercial, mais un jalon garantissant qu'un socle d'excellence homogène est partagé par l'ensemble des consultants avant leur projection en mission.

3.1.2 Le socle Java et la programmation orientée objet comme vecteurs de qualité

Le choix d’asseoir la formation initiale sur l’écosystème Java/JEE répond à des critères d’ingénierie logicielle rigoureux :

1. **Rigueur et expressivité de la Programmation Orientée Objet (POO) :** En tant que langage fortement typé et orienté objet, Java impose une discipline de modélisation stricte. Ses mécanismes fondamentaux (encapsulation, polymorphisme, héritage, interfaces, générique) permettent de transcrire fidèlement les règles et les relations des domaines métiers complexes. Cette fidélité de modélisation est indispensable à la conception d’architectures web robustes et modulaires ;
2. **Gestion avancée de la concurrence et du cycle de vie :** La maîtrise des particularités fines de la JVM (mécanismes du *Garbage Collector*, collections thread-safe, flux réactifs) est essentielle pour concevoir des applications résilientes sous forte charge et prévenir les régressions de performance ;
3. **Base fondamentale pour l’écosystème d’entreprise :** La maîtrise du langage sous-jacent (Java SE) constitue le prérequis indispensable pour appréhender avec discernement les frameworks d’entreprise tels que Spring Boot, Spring Security et Hibernate, évitant ainsi le piège du « développement par copier-coller » ou l’usage magique d’annotations sans compréhension des mécanismes sous-jacents.

3.1.3 Limites de l’outillage existant et identification des besoins

Bien qu’un outil interne d’entraînement aux certifications ait été utilisé par le passé au sein d’Oxyl, les retours d’expérience ont mis en lumière plusieurs lacunes structurelles :

- **Carence d’analyses et de métriques granulaires :** Les versions précédentes ne proposaient qu’un score brut en fin de série, sans capacité de décomposer les résultats par domaine de connaissances. Les stagiaires ne disposaient d’aucune visibilité quantifiée sur leurs points faibles spécifiques au sein d’un même programme d’examen ;
- **Absence de suivi pédagogique centralisé :** Les tuteurs et encadrants ne disposaient pas d’une vue d’ensemble consolidée pour évaluer la trajectoire de montée en compétences d’une promotion entière ni pour adapter leur accompagnement aux difficultés récurrentes ;
- **Manque d’interactivité et d’outils d’approfondissement :** En mode entraînement, la découverte d’une erreur n’était pas couplée à des mécanismes d’exploration complémentaires permettant à l’apprenant de creuser la notion théorique au-delà du corrigé statique.

3.1.4 Cahier des charges fonctionnel et innovations de la plateforme

Pour combler ces lacunes, le cahier des charges de *NewroFactory* s’est articulé autour de deux axes majeurs : l’autonomie analytique de l’apprenant et le pilotage fin par l’encadrement.

Besoins apprenants : Visualisation analytique et synergie avec les LLM

Pour maximiser l’efficacité de l’entraînement, la plateforme devait intégrer :

- **Un tableau de bord de diagnostic des compétences :** Présentation claire et hiérarchisée des pourcentages de réussite par chapitre et par série, complétée par le taux de complétion

des questions disponibles par notion. Cette double métrique permet au stagiaire d'identifier immédiatement les chapitres sous-travaillés ou mal assimilés ;

- **Deux modes d'entraînement complémentaires** : Un mode *Entraînement* interactif (feedback immédiat et explications détaillées pour chaque proposition) et un mode *Examen blanc* (chronométré, tirage étanche, conditions réelles) ;
- **Export structuré JSON pour approfondissement via LLM** : Innovation clé développée sur la plateforme, l'interface de correction en mode entraînement propose un bouton d'exportation instantané au format JSON. Ce bouton copie dans le presse-papier l'intégralité du contexte de la question (énoncé, code snippet, propositions, statut de validation, explications). L'apprenant peut ainsi soumettre directement ces métadonnées à un grand modèle de langage (LLM) afin d'obtenir des explications interactives, des contre-exemples ou des variantes pédagogiques sur la notion précise abordée.

Besoins encadrants : Gouvernance et assignation ciblée

Du côté de l'administration et du tutorat technique, les exigences comprenaient :

- **Gestion des identités et des rôles** : Distinction stricte entre les profils stagiaires et formateurs/administrateurs ;
- **Supervision globale des promotions** : Tableau de bord analytique transverse offrant aux tuteurs une visibilité sur les statistiques agrégées et individuelles de l'ensemble des stagiaires ;
- **Création et assignation de séries personnalisées** : Capacité pour l'encadrant de composer des séries d'évaluation sur-mesure et de les assigner directement au tableau de bord de stagiaires ciblés pour remédier à des lacunes précises ;
- **Administration dynamique du référentiel pédagogique** : Outils complets de gestion (CRUD) des chapitres hiérarchiques, des questions, des propositions et de leurs explications associées.

3.1.5 Objectifs d'ingénierie et exigences implicites

Le projet s'est vu confier une consigne fondatrice : **convaincre les parties prenantes et les équipes commerciales** de la pertinence de la solution en démontrant qu'elle répondait avec excellence aux standards industriels actuels.

Cette orientation imposait de facto des contraintes de réalisation implicites mais déterminantes :

- **Excellence ergonomique et design moderne** : Concevoir une interface soignée, responsive et intuitive (développée sous Angular), surpassant les outils internes traditionnels pour susciter une adhésion immédiate des utilisateurs ;
- **Robustesse et performance logicielle** : Assurer une réactivité instantanée des interactions et garantir l'étanchéité des sessions d'examen via une architecture backend Spring Core/Boot découplée ;
- **Sécurité et intégrité** : Mettre en œuvre une gestion robuste des sessions et des droits d'accès pour préserver la confidentialité des évaluations et l'intégrité des données d'examen.

3.2 Ingénierie de la plateforme : Conception d'un système de contrôle qualité des connaissances

Pour concrétiser la politique d'excellence technique d'Oxyl et standardiser le capital humain, nous avons conçu et développé *NewroFactory*, une plateforme web sur-mesure d'évaluation et de validation des compétences. Ce système vise un double objectif pédagogique et opérationnel : offrir aux apprenants un environnement d'auto-évaluation interactif aligné sur les référentiels officiels de l'industrie (tels qu'Oracle Java SE OCP et AWS Cloud Practitioner), tout en fournissant aux encadrants techniques un instrument de mesure quantitatif et objectif de la progression des stagiaires.

3.2.1 Modélisation du domaine et structuration arborescente des connaissances

La première exigence d'ingénierie a consisté à concevoir un modèle de données capable d'épouser fidèlement la granularité des référentiels pédagogiques officiels sans introduire de rigidité structurelle.

Hierarchisation des chapitres par chemin matérialisé (*Materialized Path*)

Les programmes d'examens professionnels s'organisent selon une hiérarchie stricte en modules, chapitres et sous-notions (par exemple : *Java Concurrency* → *Thread-Safe Collections* → *ConcurrentHashMap*). Pour représenter cette arborescence parent-enfant en base de données relationnelle sans subir les limites de performance des requêtes récursives profondes (*Common Table Expressions* ou jointures récursives multiples), nous avons mis en œuvre le patron de conception par **chemin matérialisé** (*Materialized Path*).

Dans l'entité `ChapterEntity`, chaque nœud stocke explicitement la chaîne hiérarchique de ses ascendants dans un champ `parentPath` (ex. `"/1/4/"`) :

```
1 @Entity
2 @Table(name = "chapter", uniqueConstraints = @UniqueConstraint(columnNames = {
3     "name", "parent_path"}))
4 @SoftDelete(columnName = "is_deleted")
5 public class ChapterEntity {
6
7     @Id
8     @GeneratedValue(strategy = GenerationType.IDENTITY)
9     private Long id;
10
11     @Column(name = "name", nullable = false)
12     private String name;
13
14     @Column(name = "parent_path")
15     private String parentPath;
16
17     @Formula("is_deleted")
18     private Boolean isDeleted;
19
20     // Getters, Setters et methodes metier
21 }
```

Listing 3.1 – Modélisation de l’entité ChapterEntity avec chemin matérialisé et suppression logique

Cette modélisation permet d’effectuer des opérations ensemblistes hautement performantes :

- L’extraction de l’ensemble des sous-chapitres descendants d’un nœud s’effectue par un simple filtre de préfixe SQL (`parent_pathLIKE' /1/4/% '`);
- L’agrégation et le décompte global des questions rattachées à un module complet et à l’ensemble de ses sous-arborescences s’exécutent en une unique passe indexée (`countQuestionsByChapterIdsAndChildren`);
- L’intégrité de l’arbre est protégée par une contrainte d’unicité composite sur (`name`, `parent_path`) et par la mise en œuvre du mécanisme de suppression logique (*Soft Delete*) via l’annotation Hibernate `@SoftDelete`.

Modélisation des questions et étanchéité des examens blancs

Les questions sont modélisées sous forme de Questionnaires à Choix Multiples (QCM) sans pénalité de points négatifs, conformes aux modalités réelles des certifications Oracle et AWS. Chaque question est rattachée à un chapitre spécifique et comporte un ensemble de propositions dont une ou plusieurs peuvent être valides.

Un enjeu méthodologique central résidait dans l’évitement du biais de mémorisation passive. Si les stagiaires s’entraînent quotidiennement sur l’ensemble de la base de questions, les examens blancs hebdomadaires du vendredi perdent leur valeur diagnostique. Pour garantir une étanchéité stricte :

1. Les questions réservées aux examens officiels sont isolées dans un chapitre racine dédié (*Examen Blanc*);
2. Les sessions d’entraînement permettent au stagiaire de sélectionner librement le chapitre cible ainsi que le volume de questions souhaité, lesquelles sont alors sélectionnées aléatoirement au sein du vivier d’entraînement;
3. Les examens blancs imposent un gabarit rigoureusement identique aux conditions réelles de passage (même volumétrie intégrale et tirage exclusif parmi les questions étanches).

Modes d’évaluation : Entraînement vs Examen

La plateforme intègre deux modes d’évaluation distincts :

- **Le mode Entraînement (formatif)** : favorise l’apprentissage itératif en fournissant une correction immédiate dès la validation de chaque question, accompagnée d’une explication technique détaillée justifiant pourquoi les alternatives erronées ont été écartées;
- **Le mode Examen (sommatif)** : simule l’épreuve officielle. Aucun feedback ni indice n’est retourné pendant le déroulement de la série. Le stagiaire ne découvre son score global qu’à l’issue de la soumission définitive, sans affichage des explications par question afin d’éviter toute fixation de réponses lors des sessions ultérieures.

3.2.2 Architecture logicielle backend et services RESTful

Le backend de *NewroFactory* a été développé en Java avec le framework Spring, selon une architecture en couches étanches respectant les principes de responsabilité unique (*Single Responsibility*

Principe - SRP) et d'inversion de dépendances.

Découpage en couches et exposition des API

L'architecture sépare rigoureusement la persistance, le domaine métier et l'exposition REST :

- **Couche Présentation / Contrôleurs (`web.rest`)** : les points d'entrée tels que `ChapterController`, `SerieController` et `UserController` valident les requêtes entrantes (via Jakarta Validation et des validateurs personnalisés comme `StartDateBeforeEndDateValidator`), invoquent les services métiers et retournent des objets de transfert typés (*DTOs*);
- **Couche Métier / Services (`service`)** : orchestre la logique d'évaluation, l'assemblage dynamique des séries aléatoires (`RandomSerieRequest`), le calcul des taux de réussite et la gestion des utilisateurs;
- **Couche Persistance (`persistence`)** : s'appuie sur Spring Data JPA et Hibernate pour abstraire l'accès aux tables relationnelles MySQL via des interfaces de dépôts typées (`ChapterRepository`, `SerieRepository`, `SerieReponseRepository`).

Observabilité et découplage transversal par Spring AOP

Dans une application d'entreprise, la journalisation (*logging*) et la gestion des erreurs ne doivent pas polluer la logique métier. Pour garantir une observabilité exhaustive et standardisée, nous avons mis en place une programmation orientée aspect (*Aspect-Oriented Programming* - AOP) via le composant `LoggingAspect`.

```
1 @Aspect
2 @Component
3 public class LoggingAspect {
4
5     @Pointcut("execution(public * com.oxyl.newrofactory.service.*(..)) || "
6             + "execution(public * com.oxyl.newrofactory.persistence.*(..))
7             || "
8             + "execution(public * com.oxyl.newrofactory.web.*(..))")
9     public void allLayersMethods() {}
10
11     @Before("allLayersMethods()")
12     public void logBefore(JoinPoint jp) {
13         getLogger(jp).info("-> Appel de : {}.{}",
14             jp.getTarget().getClass().getSimpleName(), jp.getSignature().
15             getName());
16     }
17
18     @AfterReturning(pointcut = "allLayersMethods()", returning = "result")
19     public void logAfterReturning(JoinPoint jp, Object result) {
20         getLogger(jp).info("<- Retour de : {} -> {}",
21             jp.getSignature().getName(), result);
22     }
23
24     @Pointcut("within(com.oxyl.newrofactory.persistence..*)")
25     public void persistenceMethods() {}
26
27     @AfterThrowing(pointcut = "persistenceMethods()", throwing = "exception")
28     public void translateDataAccessException(JoinPoint jp, DataAccessException
29         exception) {
30         Logger logger = getLogger(jp);
```



```

28     String className = jp.getTarget().getClass().getSimpleName();
29     String methodName = jp.getSignature().getName();
30     if (exception instanceof EmptyResultDataAccessException) {
31         logger.warn("!! [FONCTIONNEL] Element non trouve dans {}.{}() : {}"
32             ,
33             className, methodName, exception.getMessage());
34     } else {
35         logger.error("!! [CRITIQUE] Erreur SQL/Base de donnees dans
36             {}.{}() : {}" ,
37             className, methodName, exception.getMessage());
38     }
39 }

```

Listing 3.2 – Aspect transverse de logging et traduction des exceptions de persistance

Cet aspect intercepte automatiquement chaque transition inter-couches à l'entrée et à la sortie des méthodes, mesure les flux et assure une discrimination fine entre les anomalies fonctionnelles (ex. entité introuvable notifiée en avertissement) et les défaillances critiques d'infrastructure (erreurs SQL journalisées au niveau ERROR).

Moteur d'analytics et consolidation des statistiques

Pour offrir une vue claire sur l'acquisition des compétences, le système consolide deux niveaux de métriques :

- **Le score par série** : calculé à la fin de chaque questionnaire pour évaluer la performance ponctuelle sur un tirage donné ;
- **Le score agrégé par chapitre (ChapterStat)** : calculé à partir de l'historique complet des réponses d'un stagiaire, permettant d'identifier immédiatement les chapitres maîtrisés et les points de vulnérabilité conceptuelle.

Tandis que le stagiaire accède à son tableau de bord personnel, l'encadrant technique dispose d'un endpoint d'administration dédié (/api/chapters/root/withStat/user/{userId}) lui permettant de consulter les métriques individuelles de chaque collaborateur pour adapter son suivi pédagogique.

3.2.3 Gouvernance des accès, Sécurité et Interopérabilité Front/Back

En tant qu'outil interne destiné à une ESN, la gouvernance des identités et la sécurité des échanges constituent un pilier central de l'architecture.

Gestion des sessions et création déléguée des comptes

Afin de préserver la confidentialité des contenus et de garantir le rattachement hiérarchique des métriques, l'auto-inscription publique a été proscrite. La création de compte est exclusivement réservée aux encadrants et administrateurs (ROLE_ADMIN). Lorsqu'un encadrant crée le profil d'un stagiaire, ce dernier est explicitement rattaché à sa promotion dans le système. Lors de sa première connexion avec ses identifiants temporaires, le stagiaire est invité à réinitialiser son mot de passe via un composant dédié (update-password), ce qui active l'historisation de ses statistiques sous la supervision de son tuteur.

Authentification JWT par cookies sécurisés et résolution CORS

L'architecture applique un modèle sans état (*stateless*) basé sur des jetons JSON Web Tokens (JWT) :

- Pour immuniser le client contre les vulnérabilités de type *Cross-Site Scripting* (XSS), le jeton JWT n'est pas stocké dans le `localStorage` du navigateur, mais encapsulé dans un cookie HTTP sécurisé (`HttpOnly`, `SameSite=Lax`) émis par la fabrique `AuthCookieFactory` ;
- La séparation physique en phase de développement et de déploiement entre le serveur front-end Angular (port 4200) et l'API backend Spring (port 8080) a nécessité une configuration rigoureuse du mécanisme de partage de ressources entre origines multiples (*Cross-Origin Resource Sharing* - CORS).

Comme détaillé dans la classe `SecurityConfig`, l'instruction `configuration.setAllowCredentials(true)` a été combinée avec un filtrage explicite des motifs d'origine autorisés (`setAllowedOriginPatterns`) et l'autorisation des requêtes préliminaires d'évaluation (*preflight* OPTIONS), garantissant ainsi l'acheminement transparent et inviolable des cookies d'authentification sur l'ensemble des endpoints REST.

3.2.4 Conception de l'interface utilisateur et expérience apprenant (Angular)

L'interface utilisateur a été conçue sous le framework Angular, en accordant une attention rigoureuse à l'ergonomie, à la clarté visuelle et à la rapidité de prise en main.

Design System et architecture CSS

J'ai pris en charge la conception de la feuille de style globale, du design system et de la mise en page générale de l'application. L'objectif était de rompre avec l'aspect utilitaire brut souvent associé aux outils internes d'entreprise, pour proposer une interface moderne et valorisante :

- Définition d'une palette chromatique harmonieuse à base de variables CSS (couleurs sémantiques, contrastes validés selon les critères d'accessibilité WCAG) ;
- Mise en page responsive modulaire s'adaptant aussi bien aux postes de travail des développeurs qu'aux terminaux portables ;
- Conception de composants graphiques atomiques : cartes de lancement de quiz interactives (*card-quiz*), affichage de l'arborescence des chapitres avec indicateurs d'état et badges statistiques (*chapter-tree-node*), et boîte de dialogue modale de confirmation.

Parcours utilisateur et navigation interactive

Le front-end s'organise autour de parcours fluides pilotés par les services Angular et RxJS :

- **Le tableau de bord stagiaire** : synthèse visuelle présentant la progression globale, l'accès rapide aux séries d'entraînement par module et la liste des derniers examens blancs passés ;
- **Le module d'examen interactif** : affichage séquentiel des questions avec sélection dynamique des réponses, bascule contextuelle entre mode entraînement (dévoilement animé des justifications) et mode examen, puis récapitulatif global avec ventilation des scores à la soumission ;

- **Le portail d’administration** : interface réservée aux tuteurs permettant la gestion des promotions, la création des comptes stagiaires et la consultation directe des métriques individuelles.

3.2.5 Dynamique d’ingénierie collaborative, gestion des risques et retours d’expérience

Au-delà des aspects purement techniques de développement, la réalisation de *NewroFactory* a constitué une expérience humaine et organisationnelle fondatrice sur la gestion de projet logiciel en équipe.

Du développement individuel à l’intégration collective à 4

Le calendrier du projet s’est articulé en deux phases méthodologiques très contrastées :

1. **Phase 1 (Les 8 premiers sprints – 2,5 mois)** : chaque stagiaire a développé individuellement son propre backend *from scratch*, franchissant par l’épreuve les paliers successifs de l’ingénierie Java (du CLI natif et JDBC pur jusqu’à Spring Boot, Spring Security et Hibernate). Cette démarche a garanti l’alignement individuel de chacun sur les fondamentaux d’architecture ;
2. **Phase 2 (Sprint collaboratif final)** : nous avons sélectionné l’un des backends développés parmi les quatre comme socle commun. L’ensemble des quatre stagiaires a ensuite été réuni pour travailler simultanément sur le développement du front-end Angular et l’ajustement des API backend.

Difficultés rencontrées et arbitrages méthodologiques

Le passage brutal du travail en solitaire au développement simultané à quatre sur une base de code partagée a mis en lumière des défis d’ingénierie majeurs :

- **Gestion des flux Git et conflits de fusion** : n’ayant jamais été confrontés à une collaboration synchrone aussi intensive, nous avons initialement fait face à des régressions et à des superpositions de code dues à un manque de régularité dans la synchronisation locale (`git pull`) avant d’entamer de nouvelles modifications. Pour résoudre ces frictions, nous avons dû formaliser une discipline rigoureuse de gestion des branches et veiller à une synchronisation quotidienne systématique ;
- **Découpage et respect des périmètres de tickets** : nous avons appris par l’expérience l’importance cruciale de définir des tickets de travail strictement atomiques et parfaitement délimités. Lorsque le périmètre d’un ticket débordait sur les modules adjacents, le risque de collision entre développeurs augmentait de manière critique ;
- **Synchronisation des schémas de base de données** : chaque membre de l’équipe travaillant sur une instance MySQL locale, toute évolution structurelle du modèle de données (ajout de colonnes, modifications de contraintes) devait être répercutée manuellement par des scripts SQL partagés, soulignant l’importance future d’outils de migration automatisés (tels que Flyway ou Liquibase) ;
- **Déploiement Cloud** : la plateforme a été déployée sur l’infrastructure Cloud d’Amazon Web Services, en exploitant une instance virtuelle **AWS EC2** pour l’exécution des conteneurs applicatifs et un stockage **AWS S3** pour les ressources et sauvegardes.

Cette première réalisation grandeur nature a ancré des réflexes d'ingénierie essentiels : rigueur du découpage fonctionnel, conception d'architectures résilientes et standardisation des livrables. Elle a établi le socle indispensable pour aborder le second défi d'envergure du stage : la conception d'une usine logicielle agentique industrialisée et agnostique.

3.3 Choix techniques comme choix de maturité : Justification de l'architecture (Spring Core/Angular) pour une maîtrise approfondie des fondamentaux

Les choix d'architecture technologique retenus pour la plateforme *NewroFactory* procèdent d'un arbitrage réfléchi entre impératifs de formation, rigueur d'ingénierie et maîtrise pragmatique des délais de livraison. Loin d'être de simples sélections d'outils, ces choix constituent un véritable parti pris méthodologique visant la maîtrise des fondamentaux du développement logiciel moderne.

3.3.1 Démystification des couches applicatives : Le choix de Spring Core et de la configuration explicite

Alors que l'écosystème moderne privilégie l'utilisation immédiate de Spring Boot pour sa capacité à masquer la complexité derrière des starters auto-configurés, nous avons délibérément fait le choix d'édifier le backend sur le socle **Spring Core** et les bibliothèques Spring sous-jacentes de manière explicite.

Ce choix s'explique par la volonté de comprendre en profondeur la mécanique interne et l'articulation des couches composant une application web d'entreprise n-tiers. En renonçant à la « magie » des auto-configurations automatiques :

- **Maîtrise du conteneur IoC et injection de dépendances** : nous avons configuré explicitement le contexte d'application, la déclaration et le cycle de vie des Beans métier, forçant une compréhension fine du principe d'Inversion de Contrôle (*Inversion of Control* - IoC) et d'injection de dépendances ;
- **Configuration explicite des filtres et servlets** : la mise en œuvre manuelle de la chaîne de filtres de sécurité (`SecurityFilterChain`), du découplage des aspects de journalisation (`LoggingAspect`) et de la gestion des erreurs HTTP a permis d'appréhender le fonctionnement exact du moteur Web de Spring (`DispatcherServlet`) sans subir d'abstractions opaques ;
- **Appropriation de l'architecture n-tiers** : l'explicitation du passage de données entre les contrôleurs REST, les services transactionnels et la persistance relationnelle a renforcé les réflexes de découplage et d'isolation des responsabilités (*Single Responsibility Principle*).

3.3.2 Adoption d'Angular : Rigueur structurelle et opportunité de montée en compétences

Côté frontend, le choix s'est porté sur le framework **Angular**. Ce choix répond à un double objectif technique et d'apprentissage :

- **Rigueur architecturale et typage strict** : Angular impose une structure orientée composants, services et injection de dépendances particulièrement rigoureuse, véhiculée par le langage TypeScript. Ce typage strict à la compilation prémunit l'application contre toute une classe d'erreurs d'exécution et aligne la conception client sur les exigences de qualité du backend Java ;
- **Montée en compétences sur un framework d'entreprise** : par rapport à d'autres bibliothèques UI plus répandues dans le cadre académique (telles que React ou Vue.js), Angular est un framework complet mais plus exigeant. Ce projet représentait une opportunité privilégiée d'expérimenter et d'explorer une technologie robuste et largement éprouvée au sein des grands comptes industriels et des ESN.

3.3.3 Monolithe modulaire vs Microservices : Pragmatisme et focalisation sur le prototype

Sur le plan de l'architecture globale, nous avons opté pour un **monolithe modulaire** rigoureusement découpé en paquets métier. Ce choix résulte d'une décision d'ingénierie pragmatique dictée par deux facteurs majeurs :

- **Respect du calendrier et gestion du temps** : le temps imparti pour cette phase de réalisation interdisait la sur-ingénierie liée aux architectures distribuées en microservices (gestion du réseau, découplage des bases de données par service, passerelles d'API et orchestration complexe) ;
- **Focus sur le livrable fonctionnel (POC)** : l'objectif prioritaire était de concevoir et présenter une preuve de concept (*Proof of Concept* - POC) fonctionnelle, robuste, testable et de haute qualité dans les délais impartis, plutôt qu'un système sur-dimensionné. La structure monolithique modulaire garantit ainsi une cohérence transactionnelle immédiate tout en réservant la possibilité d'un découplage futur si le volume d'utilisateurs l'exigeait.

3.3.4 Écosystème outillé et composants clés du socle technique

Pour compléter ce socle d'ingénierie, plusieurs outils et bibliothèques fondamentaux ont été intégrés et articulés :

- **Apache Maven** : employé comme moteur de build standardisé et gestionnaire d'outils, assurant une gestion transparente des dépendances, la répétabilité des builds et le packaging en archive exécutable ;
- **Hibernate et JPA (*Java Persistence API*)** : assurent l'abstraction relationnelle et la correspondance objet-relationnel (ORM), permettant la manipulation fluide des entités métier (*ChapterEntity*, *QuestionEntity*) et l'exécution optimisée des requêtes ;
- **Jakarta Validation** : utilisé pour garantir la validation déclarative et systématique des DTOs en entrée d'API (*@NotNull*, *@Size*, validateurs personnalisés), assurant l'étanchéité des données saisies ;
- **JWT (*Java JWT*)** : exploité pour la création et la vérification cryptographique des jetons d'authentification JSON Web Tokens, sécurisant les échanges de session de manière stateless via des cookies HTTP sécurisés.

4 La standardisation du capital technique (Volet 2 - L'usine IA)

4.1 État de l'art : L'industrialisation du cycle de vie logiciel (SDLC) et les agents IA (SWE-bench)

L'intégration des modèles de langage de grande taille (*Large Language Models* - LLMs) au sein du cycle de vie du développement logiciel (*Software Development Life Cycle* - SDLC) représente une transition majeure dans la manière d'analyser, de concevoir et d'entretenir les systèmes d'information. Longtemps cantonnée à de l'assistance contextuelle ponctuelle, l'automatisation s'oriente désormais vers des architectures agentiques capables d'exécuter de manière autonome des sous-ensembles complexes du SDLC.

4.1.1 L'évolution du SDLC et la primauté de la génération de code

Parmi l'ensemble des phases constitutives du SDLC — analyse des besoins, modélisation architecturale, développement, revue de code, tests d'intégration et déploiement — l'étape de **génération et de modification de code** constitue le verrou le plus stratégique et le plus consommateur de ressources.

Historiquement, l'évaluation des capacités des LLMs s'appuyait sur des jeux de données synthétiques tels que HumanEval [1] ou MBPP, mesurant la génération d'une fonction isolée à partir d'une spécification courte. Bien que pertinents pour valider la compréhension algorithmique de base, ces indicateurs s'avèrent insuffisants pour représenter l'ingénierie logicielle industrielle, caractérisée par des dépôts volumineux, des dépendances inter-fichiers et la nécessité de respecter des patrons d'architecture préexistants.

L'industrialisation moderne du SDLC requiert ainsi des modèles non seulement capables de rédiger de nouvelles implémentations, mais surtout de naviguer au sein de bases de code existantes, de diagnostiquer des régressions, et de proposer des diffs directement intégrables au workflow d'intégration continue (CI/CD).

4.1.2 Évaluation agentique par SWE-bench et assistants en ligne de commande

Pour répondre aux limites des métriques de génération synthétiques, le benchmark **SWE-bench** [2] s'est imposé comme l'étalon-or scientifique pour l'évaluation des agents autonomes de génie

logiciel. Basé sur l'extraction d'issues réelles issues de projets open-source Python d'envergure, SWE-bench confronte l'agent à une description de problème et à l'état complet du dépôt Git. La réussite est conditionnée par la capacité de l'agent à générer un patch validant l'ensemble de la suite de tests unitaires du projet sans introduire de régression.

Dans une perspective d'intégration opérationnelle au court terme, le choix des interfaces d'exécution s'oriente vers des outils joignables par **ligne de commande (CLI)**, notamment :

- **Google Antigravity SDK** : permettant l'orchestration dynamique d'agents et de sous-agents autonomes spécialisés pour le suivi de tâches complexes ;
- **OpenAI Codex / CLI** : fournissant des capacités d'édition directe et de modification raisonnée de fichiers source ;
- **Claude Code CLI** : offrant des capacités avancées de raisonnement à fenêtre de contexte étendue et d'exécution d'instructions shell interactives.

L'évaluation de ces outils à l'aide de SWE-bench permet de qualifier rigoureusement leur niveau d'autonomie et leur résilience face à la complexité des tâches d'ingénierie.

4.1.3 Analyse comparative des solutions agentiques du marché

Face à la multiplication des solutions commerciales et open-source d'assistance agentique (telles que Devin, GitHub Copilot Workspace, Cursor ou SWE-agent [3]), une analyse comparative s'impose pour positionner la stratégie retenue au sein d'Oxyl.

Le Tableau 4.1 synthétise la comparaison entre l'approche d'usine logicielle agnostique développée et les solutions majeures du marché selon quatre critères fondamentaux.

TABLE 4.1 – Analyse comparative des solutions agentiques du marché vs l'Usine IA Oxyl

Solution	Intégration CLI / CI-CD	Agnosticisme LLM (Anti lock-in)	Scope d'édition code	Extensibilité des workflows
GitHub Copilot	Faible (IDE/Web)	Propriétaire (OpenAI)	Fichier / In-line	Limitée (Extensions)
Devin / SaaS proprietary	Web / Clôturé	Propriétaire / Intégré	Projet complet	Configurable (SaaS)
Cursor / Claude Code	CLI / IDE local	Multi-modèles restreint	Projet complet	Scriptable
Usine IA Oxyl (Proposée)	Native CLI & CI-CD	Agnostique (Google/OpenAI/Claude)	Multi-fichiers complet	Totale (Patrons de conception & SDK)

Cette analyse met en évidence que l'usine logicielle Oxyl privilégie l'indépendance technologique et la personnalisation fine des flux agentiques, combinant la souplesse d'exécution CLI à un contrôle total sur l'infrastructure de développement.

4.2 Théorie de l'abstraction : Étude des patrons de conception pour s'affranchir du lock-in

L'intégration d'assistants et d'agents d'intelligence artificielle au sein d'une usine logicielle industrielle confronte l'architecture à un risque majeur d'enfermement technologique (*vendor lock-in*).

Les fournisseurs de modèles de langage (LLMs) et d'outils agentiques — tels qu'Anthropic (Claude Code), Google (Antigravity SDK / Gemini) ou OpenAI (Codex / GPT) — imposent des interfaces de programmation, des formats d'outillage (*tool-calling*), des protocoles d'authentification et des modes d'exécution profondément disparates.

Pour garantir la pérennité de l'usine logicielle, son indépendance stratégique et la substituabilité immédiate d'un modèle par un autre sans réécriture du cœur applicatif, nous avons mené une étude approfondie des patrons de conception logicielle (*design patterns*) dédiés à l'abstraction.

4.2.1 Dilemme d'ingénierie et arbitrage des patrons de conception

La conception du socle d'abstraction agentique a nécessité la confrontation de plusieurs patrons classiques du *Gang of Four* (GoF) :

- **L'Adapter (Adaptateur)** : Ce patron a été retenu comme le pivot conceptuel de l'architecture. La relation entre le moteur d'orchestration de l'usine et les différents moteurs d'IA est fondamentalement une relation de **traduction** : il s'agit de convertir des requêtes canoniques d'ingénierie (prompts, outils autorisés, contextes d'exécution) vers les syntaxes d'invocation spécifiques à chaque fournisseur (flags CLI, schémas JSON d'outils, flux d'entrée standard) et d'en normaliser les retours.
- **La Strategy (Stratégie)** : Le patron Strategy a été abondamment étudié et s'hybride naturellement avec notre implémentation de l'Adapter. Le moteur applicatif interagit avec une interface générique stable (LLMProvider / AgentRunner) définissant les signatures des méthodes d'invocation. Les classes d'adaptation concrètes implémentent ces méthodes selon les spécificités de chaque modèle. Le choix du moteur à solliciter étant résolu dynamiquement à l'exécution selon le profil de la tâche, l'interchangeabilité de l'algorithme sous-jacent s'apparente à une Strategy.
- **Le Bridge (Pont)** : Le patron Bridge permet de découpler une hiérarchie d'abstractions (les types d'activités logicielles : analyse de ticket, génération de code, audit de sécurité) de multiples hiérarchies d'implémentations (les protocoles d'exécution CLI, API REST, WebSocket). Bien que séduisant, son adoption complète a été écartée au profit d'une interface d'adaptation plus directe afin d'éviter une sur-ingénierie prématurée.
- **Abstract Factory vs Singleton pour l'instanciation** : L'instanciation de l'adaptateur a fait l'objet d'un arbitrage spécifique. Si une *Factory* (fabrique) a été envisagée pour instancier dynamiquement l'adaptateur requis, nous avons opté pour la conservation d'une instance unique (**Singleton**) en mémoire durant toute la durée de vie d'un conteneur worker. L'accès à cet adaptateur s'effectue via un résolveur dédié (`getLLMProvider()`), combinant ainsi la flexibilité d'une fabrique à la cohérence d'un point d'accès persistant en mémoire.

4.2.2 Modélisation de la couche d'abstraction des moteurs d'IA

L'architecture d'abstraction projetée pour les moteurs LLM s'articule autour d'une interface canonique commune, d'un résolveur dynamique et d'adaptateurs spécialisés par fournisseur. Cette démarche reproduit fidèlement les principes d'abstraction déjà validés sur les composants de gestion de versions et de tickets.

La Figure 4.1 schématise cette organisation structurelle.

Le contrat d'interface LLMProvider définit une frontière étanche : le code métier de l'usine

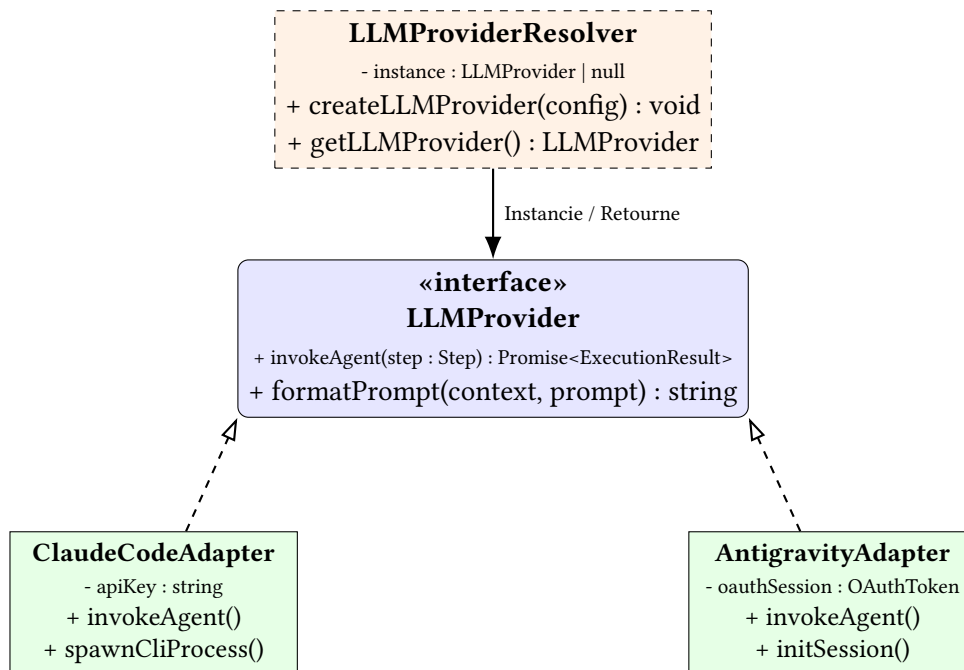


FIGURE 4.1 – Diagramme conceptuel du patron Adapter appliqué au découplage des moteurs LLM

logicielle (gestion des étapes du workflow, validation mécanique, persistance des états) n’a aucune dépendance directe envers les SDKs ou binaires propriétaires.

4.2.3 Défis d’abstraction et contraintes d’exécution conteneurisée

L’unification des moteurs d’intelligence artificielle au sein d’environnements d’exécution automatisés (conteneurs Docker isolés et sans interface graphique) met en exergue des disparités techniques majeures que l’adaptateur doit normaliser :

1. **Hétérogénéité des protocoles d’accès et d’authentification :**
 - **Claude Code CLI :** Fonctionne nativement en ligne de commande autonome. Son exécution au sein du conteneur repose sur l’injection de variables d’environnement (ANTHROPIC_API_KEY), l’invocation d’un sous-processus sans état (spawnSync) et la transmission du prompt par le flux d’entrée standard (stdin);
 - **Google AntigraVity SDK :** Conçu originellement pour opérer comme une extension intégrée à un environnement de développement interactif (IDE), son intégration dans un conteneur headless autonome nécessite une couche d’émulation de session et la gestion d’un jeton d’authentification OAuth spécifique.
2. **Gestion granulaire de l’outillage (Tool Gating) :** Tous les modèles ne disposent pas des mêmes conventions de déclaration d’outils. La couche d’abstraction normalise l’allocation des capacités selon la sensibilité de l’étape :
 - Les étapes marquées read_only (audits, revues de code) ne transmettent aux adaptateurs que les outils d’exploration et de consultation (Read, Glob, Grep) et de rapport (Write), tout en désactivant strictement l’outil de modification de code source (Edit);
 - Les étapes de développement ouvrent l’ensemble des capacités d’édition et d’exécution de commandes.
3. **Normalisation des retours et injection contextuelle de résilience :** En cas d’échec d’une étape (erreur de compilation ou régression de tests), l’adaptateur injecte de manière

transparente un bloc structuré de remédiation (*Retry Context*) en amont du prompt pour guider le modèle lors de la tentative suivante, tout en assurant l'assainissement automatique (`redactSecrets`) de toutes les sorties pour prévenir la fuite d'informations sensibles dans les journaux d'audit.

Cette conception modulaire garantit ainsi une agnosticité totale : l'ajout d'un nouveau fournisseur (par exemple un modèle open-source hébergé localement via Ollama ou vLLM) se résume à l'écriture d'une nouvelle classe implémentant `LLMProvider` sans impacter le reste du pipeline d'ingénierie.

4.3 Analyse des contraintes : Sécurité, gestion des coûts de tokens et reproductibilité des résultats

L'exploitation industrielle d'agents autonomes au sein du cycle de développement logiciel ne saurait se résumer à des considérations algorithmiques. En environnement de production, le déploiement de ces systèmes fait face à des contraintes opérationnelles strictes : la dérive financière liée à la consommation incontrôlée de tokens, la saturation des fenêtres de contexte, la préservation des secrets applicatifs et le caractère non déterministe des modèles génératifs.

Cette section détaille les mécanismes de contrôle, de résilience et de gouvernance mécanique mis en place pour garantir la viabilité économique, technique et sécuritaire de l'usine logicielle.

4.3.1 Maîtrise des coûts et des temps d'exécution : Le double verrou de réessai

L'un des risques majeurs inhérents aux systèmes agentiques réside dans le phénomène de « boucle de raisonnement infinie » (*infinite reasoning loop*). Lorsqu'un modèle se heurte à une anomalie sans parvenir à la corriger, il tend à multiplier les essais infructueux, engendrant une consommation exponentielle de tokens et un temps d'occupation critique des conteneurs.

Pour juguler ce risque, notre moteur intègre une politique de plafonnement à deux niveaux :

- **Plafond local par étape (`max_retries`)** : Chaque étape élémentaire du workflow d'un worker (par exemple la génération de code, l'analyse d'architecture ou l'exécution de la suite de tests) est assortie d'un quota strict de tentatives. Si une étape échoue (erreur de compilation, échec de linting), le moteur évalue le nombre d'essais consommés (`getRetryCount(step.id)`). Tant que `retryCount < step.max_retries`, une nouvelle tentative ciblée est autorisée ; dans le cas contraire, le traitement de l'étape est interrompu.
- **Disjoncteur global par ticket (`CircuitBreaker`)** : Au-delà des limites par étape, un compteur d'essais cumulés (`globalAttempts`) est maintenu pour l'ensemble du cycle de vie du worker sur un ticket donné. Dès que ce compteur atteint le seuil critique `maxGlobalAttempts` (défini par les limites globales de l'usine), le disjoncteur se déclenche immédiatement (*circuit breaker triggered*), interrompant définitivement le traitement afin d'éviter tout surcoût financier ou blocage de la file de tâches.
- **Routage conditionnel sur échec (`on_fail` et `max_on_fail`)** : Lorsqu'une défaillance survient, le moteur analyse et classeifie l'erreur (`classifyFailure`). Si un gestionnaire de déviation est configuré (ex. `gotodiagnostic_step`), le flux est redirigé vers une

étape spécialisée, tout en plafonnant le nombre total de déviations via `max_on_fail` pour empêcher les cycles de redirection infinis.

4.3.2 Gestion du flux de sortie et compression contextuelle : Le rôle du Manifest

L'exécution des agents génère des volumes considérables de données textuelles (traces d'exécution, diffs de code, journaux de compilation). L'injection brute et intégrale de ces flux dans les invites d'entrée (*prompts*) saturerait rapidement la fenêtre de contexte du modèle, provoquant une dégradation notable du raisonnement et un gonflement tarifaire.

Pour rationaliser ces flux de sortie, nous appliquons une stratégie de gestion asymétrique :

- **Filtrage et affichage sélectif dans le terminal** : Les sorties brutes générées par les sous-processus ne sont pas intégralement imprimées dans la console standard. Le moteur de logs tronque les réponses volumineuses en conservant uniquement le début et la fin du message (*head-and-tail truncation*), garantissant ainsi la lisibilité pour les développeurs tout en évitant la saturation des flux d'observabilité.
- **Persistance structurée dans le Manifest** : Plutôt que de conserver l'historique complet des échanges en mémoire vive, les informations vitales d'état sont persistées au sein d'un document structuré machine-readable nommé **Manifest** (`manifest.json`). Ce fichier consigne l'identifiant de l'étape en cours, le numéro de tentative (`attemptNumber`), l'étape ayant échoué (`failedStep`) ainsi qu'un extrait synthétique de l'erreur (`errorSnippet`) et une recommandation d'orientation (`suggestion`).
- **Injection minimale de contexte de réessai (*Retry Context*)** : Lors d'une reprise après incident, la fonction `buildRetryContext` n'extrait du Manifest que la charge utile essentielle liée à l'erreur précédente. Ce condensé contextuel est injecté en tête de prompt sous forme de directive ciblée (« *Previous failure category : syntax_error...* »), permettant au modèle de corriger son approche sans gaspiller de précieux tokens à relire des milliers de lignes de logs inutiles.

4.3.3 Sécurité, étanchéité des conteneurs et assainissement des secrets

L'exécution autonome d'instructions shell et la manipulation de code source par des modèles tiers exigent un cadre de sécurité intransigeant :

- **Isolation en conteneurs éphémères** : Chaque agent s'exécute au sein d'un conteneur Docker dédié, instancié à la volée avec des privilèges restreints et détruit à l'issue de la tâche. Cette conteneurisation garantit l'étanchéité totale du système hôte vis-à-vis des commandes exécutées par l'IA.
- **Purification systématique des flux (*redactSecrets*)** : Les variables d'environnement des conteneurs hébergent des identifiants sensibles (clés d'API LLM, tokens Git). Une fonction d'assainissement intercepte et masque systématiquement ces secrets au sein des flux `stdout`, `stderr` et des messages d'erreur avant toute sauvegarde dans les rapports d'audit ou affichage dans les logs.
- **Cloisonnement des permissions d'outils (*Tool Gating*)** : Conformément au principe du moindre privilège, les étapes ne nécessitant pas de modification de code (revue, vérification) se voient interdire l'accès à l'outil d'écriture de code (`Edit`), restreignant ainsi la surface d'attaque.

4.3.4 Gouvernance déterministe : L’arsenal des garde-fous mécaniques

La nature probabiliste des LLMs implique une variabilité inhérente de leurs livrables et un risque d’hallucination ou de dérive de périmètre. Pour transformer cette variabilité en une chaîne de production logicielle déterministe et industrielle, notre architecture impose une philosophie de **validation mécanique systématique**. Aucune étape n’est validée sur la seule auto-évaluation du modèle ; chaque livrable doit satisfaire un ensemble de vérifications déterministes et outillées.

Le moteur de validation mécanique déclaratif (**validateMechanical**)

Au terme de chaque étape du workflow, la fonction `validateMechanical(step)` évalue automatiquement la conformité des artefacts produits :

- **Vérification des livrables obligatoires (`outputs[] .required`)** : Contrôle physique de la présence sur le disque des fichiers de travail requis (tels que `.task/analyse.md`, `.task/plan.md` ou `.task/review.md`) ;
- **Règles d’intégrité de contenu** : Application de contraintes de longueur minimale (`min_length`) pour écarter les rapports tronqués, et validation de structure par expressions régulières (`contains`, `contains_regex`) pour s’assurer de la présence des sections obligatoires ;
- **Preuve d’activité effective (`git_diff_not_empty`)** : Vérifie formellement qu’une modification concrète a été enregistrée sur le dépôt (via `git diff HEAD`, présence de fichiers non suivis ou commits en avance sur la branche cible), interdisant à un agent de prétendre avoir finalisé une implémentation sans avoir apporté de changement au code source.

Garde-fous mécaniques spécialisés (*Mechanical Guards*)

Pour prévenir les comportements déviants caractéristiques des assistants de programmation, nous avons conçu une suite de scripts de contrôle autonomes exécutés à des jalons stratégiques du pipeline :

1. **Contrôle d’existence des fichiers sources (`check-source-files`)** : Exécuté après l’étape d’analyse, ce script extrait les chemins de fichiers mentionnés dans `.task/analyse.md` et vérifie leur existence réelle sur le système de fichiers, neutralisant immédiatement les hallucinations de modules inexistants.
2. **Respect strict du périmètre d’édition (`check-scope`)** : Afin d’éviter qu’un modèle ne modifie opportunément des fichiers non prévus, ce garde-fou compare la liste des fichiers modifiés (`.task/staged-files.txt`) aux fichiers explicitement validés dans le plan d’action (`.task/plan.md`). Toute altération hors périmètre déclenche l’arrêt immédiat du pipeline (`halt_if`).
3. **Étanchéité des suites de tests (`check-tests-scope`)** : Lors de l’étape de rédaction de tests (`code-tests`), l’agent a l’interdiction formelle de modifier le code source de l’application sous `src/`. Le script compare un instantané pré-tests (`staged-files-pre-tests.txt`) aux modifications finales pour garantir que seuls des fichiers sous `tests/` ont été ajoutés ou édités, empêchant le modèle d’altérer le code métier pour « faire passer » artificiellement les tests.
4. **Protection de la chaîne d’approvisionnement (`check-new-deps`)** : Analyse le fichier `package.json` par rapport à la branche de base (`dev`) et bloque le pipeline si de nou-

velles dépendances tierces ont été introduites sans validation explicite préalable, prévenant l'inflation des dépendances et les vulnérabilités de sécurité associées.

5. **Intégrité de la qualité logicielle (check-suppressions)** : Audite le diff Git pour s'assurer que le modèle n'a pas contourné les erreurs de compilation ou de typage en injectant des annotations d'inhibition (telles que `eslint-disable`, `@ts-ignore`, `@ts-expect-error`, `@SuppressWarnings` ou `NOSONAR`).

4.3.5 Reproductibilité, métriques et perspective du patron Observer

L'agrégation de l'ensemble de ces garde-fous permet de convertir le comportement probabiliste des LLMs en un flux d'ingénierie prédictible et auditable.

Afin de consolider à grande échelle ces métriques de santé et de performance, nous prévoyons l'intégration d'un composant d'observabilité fondé sur le patron de conception **Observer** (Observateur). Ce composant écouterait les événements émis par les superviseurs et les conteneurs workers, permettant de mesurer en temps réel :

- La consommation cumulée de tokens et les coûts financiers associés par projet et par ticket ;
- La durée moyenne de cycle et la répartition temporelle par étape ;
- Le taux de passage au premier essai (*First-Time Pass Rate*) et la typologie des erreurs rattrapées par les gardes mécaniques.

Ce tableau de bord constituerait l'outil central de pilotage de la rentabilité et de la robustesse globale de l'usine logicielle.

5 La mise en œuvre de l’infrastructure agnostique

5.1 Architecture du moteur agnostique : Le découplage des couches Git et des couches LLM

La concrétisation opérationnelle de l’usine logicielle autonome développée au sein d’Oxyl repose sur un impératif architectural majeur : interconnecter de multiples briques hétérogènes de l’écosystème de développement (gestionnaires de backlogs, forges de code, environnements d’exécution locaux et assistants d’intelligence artificielle) tout en garantissant un agnosticisme technologique complet, une gestion concurrente robuste et une étanchéité de sécurité absolue.

L’intégralité du moteur d’orchestration (Superviseur et Worker) a été conçue et implémentée en **TypeScript** sous l’environnement Node.js. TypeScript apporte la garantie d’un typage statique nominal rigoureux — étendu par des types opaques étiquetés (*Branded Types*) — évitant les erreurs de manipulation de jetons ou d’identifiants au moment de la compilation, tout en exploitant la puissance du modèle asynchrone non-bloquant de Node.js pour piloter les sous-processus et les flux d’entrées/sorties.

5.1.1 Conception modulaire et contrats d’interface : BacklogProvider et CodeBaseProvider

Afin d’éradiquer tout couplage fort entre la logique d’orchestration et les forges logicielles, l’architecture sépare strictement les opérations de gestion du travail et les opérations de manipulation de code au travers de deux contrats d’interface distincts :

- **L’interface BacklogProvider (Gestion du flux de travail)** : Définie dans le sous-système superviseur (`src/supervisor/providers/backlog-provider.ts`), cette interface abstrait les interactions avec les outils de ticketing. Elle formalise les opérations de découverte, de réservation et de traçabilité des tickets :
 - `fetchReadyTickets(filter, priorities)` : interroge la forge pour extraire les tickets ouverts portant les labels d’éligibilité (ex. `factory::ready`), ordonnés selon les priorités ;
 - `claimTicket(ticketId)` : applique le protocole de réservation en apposant un label de verrouillage spécifique à la machine (ex. `factory::claimed-by-<machine>`) afin de garantir l’exclusion mutuelle ;
 - `addLabels`, `removeLabels`, `updateLabels` : orchestrent les transitions d’état du ticket tout au long du cycle de résolution ;
 - `postComment(ticketId, body)` et `updateComment` : publient et mettent à jour les comptes-rendus de diagnostic, les métriques d’exécution et les liens d’intégration ;

- `getRelatedMergeRequests(ticketId)` : identifie les demandes de fusion déjà associées pour prévenir les ré-exécutions superflues.

L'adaptateur de référence implémenté est `GitLabBacklogAdapter`, avec une extension naturelle prévue vers **GitHub Issues**, **Gitea** et **Jira**.

- **L'interface `CodeBaseProvider` (Gestion des dépôts et des fusions)** : Localisée côté worker (`src/worker/providers/codebase-provider.ts`), cette interface unifie la création des demandes de fusion de code via une méthode canonique unique :
 - `handleMergeRequest(params)` : prend en charge de façon atomique la création ou la récupération d'une *Merge Request* (GitLab) ou *Pull Request* (GitHub/Gitea/Bitbucket) en transmettant la branche source, la branche cible, le titre et la description générée (renvoie un `Promise<MergeRequestResult>`).

Le moteur dispose déjà d'adaptateurs opérationnels pour **GitLab** (`GitLabCodeBaseAdapter`) et **Gitea** (`GiteaCodeBaseAdapter`), prêts à accueillir GitHub et Bitbucket.

Grâce à cette séparation, un projet peut être librement configuré pour consommer son backlog sur un outil (ex. Jira ou GitLab Issues) tout en publiant son code et ses revues sur un autre (ex. GitHub ou Gitea).

5.1.2 Typage nominal étiqueté (*Branded Types*) et résolution dynamique des instances

Pour éliminer les risques de fuites ou de confusion entre différents secrets applicatifs lors des appels d'API, le moteur exploite le mécanisme de typage nominal par étiquetage (*Branded Types*) au sein de `src/shared/providers/types.ts` :

```

1 declare const __brand: unique symbol;
2 export type Branded<T, B> = T & { [__brand]: B };
3
4 export type GitlabToken = Branded<string, 'GitlabToken'>;
5 export type GiteaToken = Branded<string, 'GiteaToken'>;
6
7 export type CodeBaseAccessInfo =
8   | { providerName: 'gitlab'; url: string; projectId: number; token:
9     GitlabToken }
10  | { providerName: 'gitea'; url: string; owner: string; repo: string; token:
11    GiteaToken };

```

Listing 5.1 – Modélisation des types étiquetés et des contrats d'accès aux providers

L'instanciation des adaptateurs est pilotée par des résolveurs dédiés (`BacklogProviderResolver` et `CodeBaseProviderResolver`) qui appliquent le patron **Registry / Singleton** :

- Au démarrage du superviseur, `createBacklogProvider()` lit la configuration du projet (`backlogAccessInfoList`) et instancie l'adaptateur adéquat;
- Le point d'accès global `getBacklogProvider()` permet aux modules d'orchestration de consommer les services de backlog sans instanciation redondante;
- Côté worker, `createCodeBaseProvider(accessInfo)` résout dynamiquement le fournisseur de code à partir des informations de connexion transmises.

5.1.3 Cycle opérationnel : Isolation par *Git Worktrees* et suivi des tâches

Plutôt que d'effectuer un clonage intégral et coûteux du dépôt Git pour chaque ticket, le superviseur d'Oxyl (SupervisorV5) met en œuvre une gestion concurrente légère et hautement performante fondée sur les **arborescences de travail Git (*Git Worktrees*)** :

1. **Boucle de surveillance et réservation atomique (*pollCycle*)** : Le Superviseur exécute une boucle périodique sécurisée :
 - Il vérifie le verrou de limite de requêtes (*checkRateLimitLock*) pour respecter les quotas d'API des forges ;
 - Il libère les verrous orphelins ou expirés laissés par d'éventuels processus interrompus (*releaseExpiredClaims*) ;
 - Il détecte les tickets éligibles via *fetchReadyTickets()*, réserve le ticket par étiquetage (*claimTicket()*), puis déclenche l'instanciation du worker.
2. **Création de l'espace de travail isolé (*setup-worktree.ts*)** : Pour chaque tâche prise en charge :
 - Une branche dédiée (ex. *feature/ticket-104*) et un répertoire de travail isolé sous *.worktrees/<nom-branche>* sont créés via *createWorktree()* ;
 - Un dossier d'échange *.task/* est initialisé avec un fichier de description de tâche *manifest.json* et automatiquement ajouté au *.gitignore* local (*ensureTaskGitignore*) ;
 - Si des dépendances sont détectées dans le projet cible, une installation propre (*npmci*) est exécutée au sein du *worktree*.
3. **Surveillance en vol et réattachement (*In-Flight Tracking*)** : Le superviseur enregistre les identifiants de processus (PIDs) et les identités d'exécution (*RunIdentity*) dans une table de suivi persistée (*in-flight-store.ts*). Un émetteur de signal de vie (*heartbeat*) garantit la détection des anomalies, tandis que la fonction *reattachSurvivors()* permet au superviseur de reprendre le suivi des workers actifs en cas de redémarrage inopiné.

5.1.4 Exécution du pipeline agentique et publication déterministe de la MR

Une fois l'environnement de travail préparé, le processus Worker (*src/worker/worker.ts / Launcher V5*) prend le relais pour dérouler la séquence d'ingénierie :

- **Confinement strict de l'agent IA (*invokeAgent*)** : L'agent d'intelligence artificielle (Claude Code CLI / Antigravity) est invoqué par sous-processus via son flux d'entrée standard (*stdin*) en lui restreignant strictement les outils autorisés (*--allowed-tools*). Il n'opère que sur les fichiers locaux du *worktree* ;
- **Assainissement et observabilité (*redactSecrets & truncateForLog*)** : Toutes les traces d'exécution de l'agent font l'objet d'un masquage systématique des clés et secrets. Les journaux volumineux sont condensés par une troncature symétrique (*head-and-tail truncation*) préservant les diagnostics initiaux et finaux ;
- **Disjoncteur global (*CircuitBreaker*)** : Le worker maintient un compteur d'essais cumulés (*globalAttempts*). Si le seuil critique (*maxGlobalAttempts=25*) est franchi suite à des échecs répétés, le disjoncteur interrompt immédiatement le traitement pour prévenir tout gaspillage de ressources ;
- **Gouvernance mécanique et script *create-mr.ts*** : Une fois le code validé par les garde-fous déterministes (*validateMechanical*), le worker n'utilise **en aucun cas l'IA**

pour publier le travail. Il exécute un script TypeScript autonome (`src/worker/providers/create-mr.ts`) qui récupère les variables d'environnement injectées (`USINE_CODEBASE_TOKEN`), instancie le `CodeBaseProvider` et déclenche l'ouverture de la Merge Request sur la forge avec les références au ticket d'origine.

5.1.5 Schéma d'architecture globale du moteur agnostique

La Figure 5.1 illustre l'architecture globale du moteur agnostique, mettant en évidence le découplage des forges, l'isolation par Git Worktrees et l'étanchéité absolue entre la zone d'intervention de l'IA et les mécanismes d'intégration déterministes.

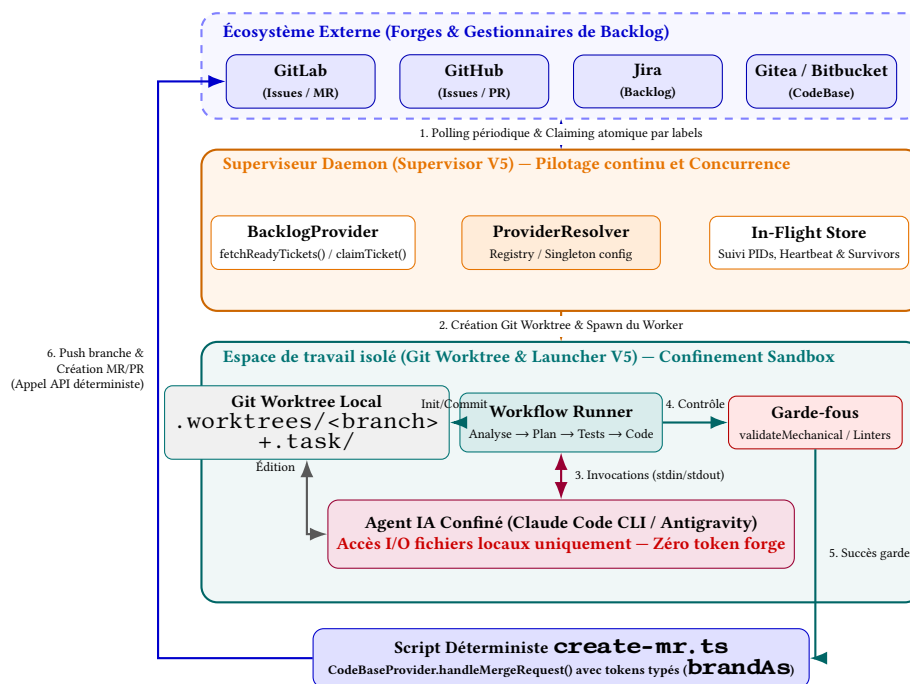


FIGURE 5.1 – Architecture globale du moteur agnostique : découplage des forges, isolation par Git Worktrees et étanchéité de l'agent IA

5.2 Stratégies de normalisation : Uniformisation des API hétérogènes (GitLab/GitHub/Gitea/LLMs)

L'efficacité et la pérennité d'une usine logicielle autonome reposent sur sa capacité à fonctionner de manière totalement agnostique vis-à-vis des outils tiers. Dans un écosystème d'entreprise, les équipes utilisent des forges hétérogènes (GitLab, GitHub, Gitea, Bitbucket), des gestionnaires de backlog variés (Jira, GitLab Issues, GitHub Projects) et différents assistants d'intelligence artificielle (Antigravity/Agy, OpenAI Codex CLI, Claude Code CLI).

Face à cette diversité, l'architecture d'Oxyl implémente une stratégie de normalisation reposant sur trois piliers :

1. Un **modèle de données canonique pivot** couplé à des *Data Mappers* pour abstraire les forges et leurs asymétries de protocole ;

2. Une **couche d'adaptation d'agents CLI** standardisant l'invocation, la surveillance et l'interchangeabilité des moteurs d'IA;
3. Un **protocole pivot d'échange documentaire** basé sur des artefacts Markdown structurés (`.task/*.md`), constituant l'espéranto entre l'orchestrateur déterministe et les agents cognitifs.

5.2.1 Modèle pivot canonique : Unification des entités métier (*Data Mapper*)

Chaque forge logicielle manipule des structures de données et des protocoles d'accès divergents. Par exemple, GitLab manipule les labels directement sous forme de chaînes textuelles (`string[]`) et identifie les projets par un ID numérique ou un chemin encodé, tandis que Gitea requiert une résolution préalable des labels sous forme d'identifiants numériques internes (`number[]`) et segmente les dépôts par couple `owner/repo`.

Pour isoler la logique métier du superviseur et du worker de ces disparités, nous appliquons le patron de conception **Canonical Data Model** (Modèle Pivot) associé à des **Data Mappers** bidirectionnels (voir Listing 5.2).

```

1 // 1. àModle canonique pivot pour les tickets de backlog
2 export type Ticket = {
3   id: number;
4   projectId: number | string;
5   title: string;
6   description: string;
7   labels: string[];
8   state: 'open' | 'closed';
9   priority: string;
10 };
11
12 // 2. àModle canonique pour les demandes d'éintégration (MR / PR)
13 export type MergeRequestResult = {
14   id: number;
15   iid: number;
16   url: string;
17   sourceBranch: string;
18   targetBranch: string;
19   state: 'opened' | 'closed' | 'merged';
20 };
21
22 // 3. Contrat d'adaptation Backlog (Mapping bidirectionnel)
23 export interface BacklogProvider {
24   fetchReadyTickets(filter: TicketFilter, priorities: PriorityConfig): Promise<Ticket[]>;
25   claimTicket(ticketId: number): Promise<boolean>;
26   addLabels(ticketId: number, labels: string[]): Promise<void>;
27   removeLabels(ticketId: number, labels: string[]): Promise<void>;
28   postComment(ticketId: number, body: string): Promise<void>;
29   getRelatedMergeRequests(ticketId: number): Promise<RelatedMergeRequest[]>;
30 }

```

Listing 5.2 – Modélisation des types pivots abstraits et du contrat d'adaptation

Chaque adaptateur concret (`GitLabBacklogAdapter`, `GiteaCodeBaseAdapter`) encapsule la complexité de sérialisation et de désérialisation :

- **Désérialisation ascendante** : Lorsqu'un adaptateur interroge une API externe, il mappe le payload brut (JSON GitLab REST ou Gitea Swagger) en une entité conforme au type pivot `Ticket` ;
- **Sérialisation descendante** : Lors des opérations de mutation (création de MR, mise à jour de labels), l'adaptateur traduit les commandes unifiées dans le format exigé par la cible. C'est l'adaptateur Gitea qui prend en charge la conversion transparente des noms de labels en IDs Gitea sans polluer le code de l'orchestrateur.

5.2.2 Abstraction et interchangeabilité des moteurs agentiques (*Agent-Provider*)

Plutôt que de coupler l'usine logicielle aux SDKs d'APIs brutes de chaque fournisseur de LLM (Anthropic API, OpenAI API, Google AI), nous avons fait le choix stratégique d'interagir avec les **CLI agentiques autonomes** : **Agy (Antigravity CLI)**, **OpenAI Codex CLI** et **Claude Code CLI**.

Ce choix offre des avantages déterminants :

- **Délégation de la gestion de contexte** : Les CLI intègrent nativement les optimisations d'historique de conversation, la gestion dynamique de la fenêtre de contexte et la compaction automatique des messages ;
- **Gouvernance centralisée des accès** : L'usine est hébergée sur l'infrastructure d'Oxyl où les comptes et quotas de tokens sont mutualisés et sécurisés. La sélection du CLI agentique est configurable dynamiquement au sein du fichier `config.json` de l'usine, ouvrant la voie à une architecture multi-tenant où chaque client pourra brancher son propre CLI et ses propres clés d'authentification ;
- **Isolation des autorisations d'outils (*Tool Sandboxing*)** : Le worker alloue dynamiquement les outils autorisés (`--allowed-tools`) en fonction de la phase en cours (voir Listing 5.3).

```

1 export function getAllowedTools(step: Step): string {
2   const baseTools = [ 'Read', 'Write', 'Glob', 'Grep', 'Bash', 'Agent' ];
3   // L'outil de édition in-situ ('Edit') est strictement éverrouill
4   // lors des étapes d'analyse, de planification ou de revue
5   if (!step.read_only) {
6     baseTools.push( 'Edit' );
7   }
8   return baseTools.join( ',' );
9 }

```

Listing 5.3 – Génération dynamique du contexte et allocation des permissions d'outils

Pour prévenir les dérives d'exécution et les gaspillages financiers liés à des agents bloqués en boucle infinie, deux mécanismes déterministes sont intégrés :

1. **Signal de vie périodique (*Heartbeat*)** : Le superviseur émet régulièrement un commentaire structuré sur le ticket (`buildClaimMetaComment`) attestant de la progression du worker. L'absence prolongée de signal permet de détecter les processus orphelins et de recycler les verrous ;
2. **Garde-fou temporel et disjoncteur (*Circuit Breaker*)** : Un temps d'exécution maximal est alloué à chaque étape du pipeline, couplé à un plafond global de tentatives (`maxGlobalAttempts=25`) au-delà duquel l'exécution est suspendue avec notification d'échec motivée.

5.2.3 Protocole pivot d'échange documentaire : Les artefacts Markdown (`.task/* .md`)

Pour orchestrer la communication entre les composants déterministes (écrits en TypeScript) et les agents probabilistes (les LLMs pilotés par CLI), l'usine utilise le langage **Markdown** comme format d'échange universel.

À chaque instantiation de tâche dans un *Git Worktree*, un répertoire d'échange dédié `.task/` est créé à la racine du projet. Ce répertoire contient des documents standardisés jouant le rôle de contrats d'interface :

TABLE 5.1 – Typologie et rôle des artefacts Markdown au sein de l'espace `.task/`

Fichier Artefact	Étape Productrice	Rôle et Contenu Normalisé
<code>ticket.md</code>	Superviseur (Init)	Synthèse normalisée du besoin : titre, description nettoyée, critères d'acceptation et contraintes de branches.
<code>plan.md</code>	Agent (plan-ticket)	Contrat d'implémentation : analyse de cause racine, périmètre exact des fichiers à créer/éditer et stratégie de tests.
<code>staged-files.txt</code>	Garde-fous / Git	Liste brute des fichiers modifiés, utilisée par <code>check-scope</code> pour valider la conformité avec <code>plan.md</code> .
<code>feedback.md</code>	Worker (Validation)	Rapport normalisé d'erreurs de compilation, de linter ou de tests unitaires injecté dans le prompt lors des boucles d'auto-correction.
<code>review.md</code>	Agent (review-code)	Rapport d'audit de qualité logicielle, d'adhérence aux conventions de nommage et d'absence de régressions.

Cette normalisation documentaire garantit une lisibilité maximale pour l'IA, facilite l'auditabilité pour les développeurs humains qui consultent les branches, et supprime tout couplage direct avec un schéma propriétaire.

5.3 Validation de la chaîne de valeur : Faisabilité de bout-en-bout et stratégie d'arbitrage des LLMs

L'évaluation de l'infrastructure agnostique s'articule autour de deux impératifs complémentaires : d'une part, la démonstration empirique de son bon fonctionnement de bout-en-bout au travers de cas d'usage réels et factices ; d'autre part, la formalisation d'un cadre d'arbitrage méthodologique permettant d'orienter le choix futur des modèles de langage (LLMs) au fur et à mesure des évolutions de l'état de l'art.

5.3.1 Validation opérationnelle et retours d'expérience sur projets réels

La chaîne de valeur automatisée a été éprouvée avec succès selon une démarche progressive, allant d'un environnement bac à sable contrôlé jusqu'au déploiement sur des projets industriels d'Oxyl :

1. **Preuve de concept sur environnement Gitea (Projet de base de connaissances) :** La première étape de validation a consisté à éprouver l'adaptateur `GiteaCodeBaseAdapter` sur un projet factice spécifiquement créé pour tester l'usine. Ce projet visait le développement complet d'une plateforme web de centralisation et de partage de connaissances, développée sous le framework **Angular**.

Ce banc d'essai a permis de valider la faisabilité technique de la boucle intégrale : découverte de tickets, réservation par étiquetage, initialisation dynamique de *Git Worktrees*, confinement de l'agent CLI et émission déterministe de *Pull Requests* sur Gitea sans aucune anomalie de synchronisation.

2. **Déploiement sur des projets en production chez Oxyl :** Forte de ces résultats, l'usine logicielle a ensuite été mobilisée sur des projets à haute valeur ajoutée menés pour des clients et partenaires :
 - **Projet Naskal :** Conception et développement d'une plateforme e-commerce dédiée à la commercialisation d'analyses de marché et de veille stratégique dans le secteur des technologies de l'information (IT);
 - **Projet BPI :** Développement d'une application interne d'analyse financière et de pilotage de projets pour le compte de la banque publique d'investissement (BPI).

Sur ces différents projets, l'usine logicielle a pris en charge l'intégralité du cycle de réalisation des fonctionnalités et des corrections de code, démontrant une maturité et une stabilité opérationnelle telles que sa généralisation à l'ensemble des futurs projets de l'entreprise est désormais enclenchée.

5.3.2 Repositionnement du rôle de l'ingénieur : Le paradigme *Human-in-the-Loop*

L'automatisation intégrale de l'implémentation par l'usine logicielle transforme en profondeur le rôle des développeurs humains au sein de l'équipe :

- **De la rédaction manuelle à la supervision experte :** Les ingénieurs ne passent plus leur temps à coder des briques répétitives ou des implémentations de routine; ils interviennent exclusivement en phase terminale lors de la **revue de code (Code Review)** sur les Merge Requests ouvertes par l'usine;
- **Exhaustivité des cas d'usage :** Le relecteur humain vérifie que l'ensemble des scénarios nominaux et des cas limites ont été rigoureusement pris en compte et couverts par des tests automatisés pertinents;
- **Adhérence aux principes Clean Code :** L'attention humaine se concentre sur l'élégance architecturale, la maintenabilité à long terme, la lisibilité et l'expressivité du nommage des variables, méthodes et modules.

Ce modèle de collaboration homme-machine garantit un débit de livraison élevé sans jamais concéder sur les standards d'excellence logicielle de l'organisation.

5.3.3 État du déploiement des moteurs LLM et impératif d'agnosticisme

Dans la configuration opérationnelle actuelle, l'ensemble des projets tourne avec le CLI agentique **Claude Code** (fondé sur les modèles Claude 3.5 et 3.7 Sonnet d'Anthropic), reconnu pour sa précision dans la manipulation multi-fichiers et la pertinence de ses raisonnements contextuels. En parallèle, l'intégration de deux autres moteurs majeurs est activement développée au sein du socle d'abstraction :

- **Google Antigravity SDK** : Pour exploiter l’orchestration multi-agents et les capacités d’analyse de Gemini à très large fenêtre de contexte ;
- **OpenAI Codex / CLI** : Pour bénéficier des capacités de synthèse et d’édition rapide des modèles GPT.

L’état de l’art des modèles de langage évolue à un rythme mensuel : aucun fournisseur ne détient un avantage absolu et pérenne sur l’ensemble des compétences de génie logiciel (diagnostic d’erreurs, refactorisation, synthèse front-end, algorithmique complexe).

L’agnosticisme technique du moteur d’Oxyl constitue donc un choix stratégique d’optionnalité : il offre la flexibilité immédiate de basculer d’un moteur à l’autre selon les forces de chaque modèle, les variations de tarification des API et les impératifs de confidentialité des clients.

5.3.4 Matrice et critères d’arbitrage pour le benchmarking futur

Afin d’évaluer scientifiquement et d’arbitrer le choix des moteurs d’IA au fur et à mesure de l’intégration de nouveaux CLI, une grille d’évaluation multicritères a été formalisée. Le Tableau 5.2 détaille les indicateurs clés de performance (KPIs) qui serviront de base aux futurs protocoles de benchmarking comparatifs.

Cette matrice permettra de conduire des campagnes d’évaluation objectives sur des jeux de tickets standardisés, assurant ainsi que l’usine logicielle sélectionne toujours le moteur le plus performant et le plus rentable pour chaque type de projet.

TABLE 5.2 – Matrice multicritère d'évaluation et d'arbitrage des moteurs d'IA agentiques

Dimension d'analyse	Indicateur clé (KPI)	Objectif opérationnel & Méthode de mesure
Exactitude & Qualité	Taux de réussite au 1 ^{er} tour	Pourcentage de tâches passant l'ensemble des tests unitaires et linters dès la première tentative sans boucle d'auto-correction.
	Fidélité au plan d'implémentation	Adhérence stricte entre les modifications effectives (<code>staged-files.txt</code>) et le périmètre validé dans <code>plan.md</code> (anti <i>scope creep</i>).
Autonomie & Résilience	Vitesse de convergence en correction	Nombre moyen d'itérations nécessaires dans la boucle de rétroaction <code>feedback.md</code> pour résoudre une erreur de compilation ou de test.
	Taux de déclenchement du disjoncteur	Fréquence d'atteinte du seuil d'essais (<code>maxGlobalAttempts=25</code>) nécessitant une escalade humaine.
Efficience & Coûts	Consommation moyenne de tokens	Nombre moyen de tokens consommés par ticket résolu (mesure de l'efficacité de la gestion de contexte).
	Temps moyen d'exécution	Durée totale écoulée entre la réservation du ticket et la création de la Merge Request.
Maintenabilité humaine	Taux d'acceptation des MRs	Pourcentage de Merge Requests validées par les développeurs sans demande de refactorisation majeure.
	Indice de lisibilité (<i>Clean Code</i>)	Clarté du code généré, adéquation du nommage et absence de duplication constatées lors de la revue.

6 Analyse critique et Impact systémique

6.1 Synergie entre les deux volets : Comment le socle de compétences facilite la maintenance et l'évolution de l'usine logicielle

L'industrialisation de la chaîne de production logicielle chez Oxyl ne repose pas sur une juxtaposition étanche entre la montée en compétences des collaborateurs (Volet 1) et l'automatisation par agents intelligents (Volet 2), mais sur une véritable symbiose opérationnelle et méthodologique. L'efficacité à long terme d'une usine logicielle assistée par l'intelligence artificielle est indissociable de la maturité technique des ingénieurs qui la pilotent, la contraignent et l'enrichissent au quotidien.

6.1.1 La boucle de rétroaction bidirectionnelle et le déplacement de la charge cognitive

L'intégration de l'usine logicielle transforme en profondeur la nature du travail de l'ingénieur. En déléguant à l'agent IA l'écriture du code de commodité (*boilerplate*), la génération des squelettes de classes ou la mise en œuvre de transformations directes, le développeur est libéré des tâches répétitives à faible valeur ajoutée.

Toutefois, cette automatisation ne diminue en aucun cas l'effort intellectuel requis ; au contraire, elle **déplace la charge cognitive vers le haut de la chaîne de valeur**. Le temps autrefois consommé par la frappe mécanique du code est désormais intégralement réinvesti dans des activités de conception stratégique :

- **La réflexion architecturale** : définition du découpage modulaire, respect des frontières de domaines, gestion des contrats d'interfaces et anticipation de la scalabilité des composants ;
- **L'application rigoureuse du *Clean Code*** : garantie de la lisibilité, minimisation du couplage, optimisation de la cohésion et préservation de la maintenabilité du patrimoine applicatif ;
- **L'arbitrage pragmatique de l'exécution** : lorsqu'un modèle opère de manière fluide et génère le comportement attendu dès la première passe, le gain de productivité est immédiat. En revanche, si l'usine nécessite trois itérations successives pour accomplir une tâche élémentaire en dérivant vers des corrections bancales, le coût cognitif de supervision et de contextualisation devient supérieur au temps de codage manuel. L'ingénieur doit alors immédiatement identifier ce point de bascule pour reprendre la main et coder directement la solution.

Ce déplacement de rôle confère au développeur une posture d'**ingénieur-architecte et arbitre**,

garant de la cohérence systémique du projet.

6.1.2 L'exigence accrue d'un socle théorique face aux heuristiques de l'IA

L'usage intensif des modèles de langage en ingénierie logicielle met en lumière un paradoxe fondamental : **l'automatisation du code n'abaisse pas le niveau d'expertise requis, elle l'élève.**

Lors d'une revue de code standard entre pairs humains, la démarche intellectuelle de l'auteur est généralement lisible, car elle suit des schémas mentaux, des conventions d'équipe et une logique déductive partagée. À l'inverse, un agent IA génère du code par inférence probabiliste. Il tend fréquemment à proposer des solutions fonctionnelles mais alambiquées, inutilement complexes ou masquant des régressions insidieuses. Analyser et évaluer ce type de code requiert une **finesse d'analyse supérieure** à celle nécessaire pour une revue classique.

C'est ici que le socle acquis lors de la préparation aux certifications industrielles (notamment l'OCP Java SE) révèle toute sa valeur :

1. **Compréhension fine des mécanismes internes** : sans une maîtrise rigoureuse de la gestion mémoire, du modèle de typage strict, de la portée des variables, des subtilités du multithreading ou de la structure des collections, il est impossible de déceler les anti-patterns ou les failles subtiles introduites par l'agent ;
2. **Capacité de guidage et d'alignement** : l'amélioration continue des agents repose sur la qualité des spécifications, du contexte injecté et des consignes transmises. Un principe fondamental d'ingénierie s'impose : *il est impossible de guider une IA vers une solution propre et optimale si l'ingénieur lui-même ne possède pas des standards de qualité et une discipline de développement supérieurs à ceux du modèle.*

Le socle théorique standardisé constitue ainsi le rempart indispensable contre l'accumulation de dette technique générée par la machine.

6.1.3 Transversalité des paradigmes : du socle Java OCP à l'architecture TypeScript de l'usine

La synergie entre les deux volets illustre également la portée universelle des fondamentaux logiciels au-delà des écosystèmes technologiques particuliers.

Bien que le parcours de certification et la plateforme de quiz (Volet 1) soient principalement structurés autour du langage Java, l'infrastructure de l'usine logicielle (Volet 2) a été développée en TypeScript. Malgré cette divergence apparente de syntaxe et d'environnement d'exécution, les passerelles conceptuelles sont directes et omniprésentes :

- **L'assimilation de la généricité** : les notions approfondies de types paramétrés, de bornes génériques et d'inférence de types étudiées pour l'OCP en Java ont permis d'appréhender avec aisance les fonctionnalités de typage générique avancé de TypeScript au sein du moteur de l'usine ;
- **La généralisation des patrons de conception (*Design Patterns*) et des principes SOLID** : les mécanismes de découplage (usines abstraites, stratégies, adaptateurs, observateurs) et les règles de responsabilité unique ne dépendent d'aucun langage spécifique. Ils four-

nissent un cadre de pensée universel qui a guidé la conception modulaire des orchestrateurs de l'usine tout en assurant l'interopérabilité des composants. Cette transférabilité démontre que l'investissement consenti dans la maîtrise des abstractions fondamentales offre une agilité pérenne, permettant aux collaborateurs d'intervenir avec la même rigueur sur les applications d'entreprise traditionnelles que sur les architectures émergentes d'outillage automatisé.

6.2 Impact économique et technologique : ROI, réduction du vendor lock-in, gains de productivité et attractivité client

L'industrialisation conjointe du capital humain et du capital technique dépasse la simple optimisation méthodologique : elle constitue un levier économique et stratégique direct pour le modèle d'affaires d'Oxyl. En articulant la montée en compétences certifiée et l'automatisation agentique agnostique, l'entreprise transforme sa structure de coûts, accélère ses cycles de livraison et renforce son attractivité auprès des donneurs d'ordre les plus exigeants.

6.2.1 Modélisation du retour sur investissement (ROI) et gains de productivité

L'évaluation de la rentabilité de l'usine logicielle (Volet 2) repose sur une comparaison directe entre le coût marginal d'inférence de la machine et le coût horaire d'ingénierie humaine.

Dans une chaîne de développement traditionnelle, le traitement d'une tâche de maintenance corrective ou de commodité (création de structures de données, implémentation de squelettes de services, écriture de tests unitaires d'intégration ou correction de régressions ciblées) mobilise un consultant durant 1 à 3 heures. En considérant le taux journalier moyen (TJM) usuel du marché pour un ingénieur d'études, le coût unitaire d'un tel traitement humain oscille entre 70 € et 250 €.

À l'inverse, l'exécution complète d'un pipeline de résolution par notre usine logicielle agentique — comprenant l'analyse du dépôt Git, la navigation dans l'arbre syntaxique, la planification, la génération du patch et la validation par les suites de tests — consomme en moyenne entre 50 000 et 200 000 jetons (*tokens*) selon la complexité du contexte. Au tarif en vigueur des modèles d'inférence avancés, ce cycle automatisé représente un coût machine direct compris entre 0,20 € et 1,50 € par ticket. Même en intégrant le temps humain indispensable de supervision, de relecture critique et de validation finale (10 à 15 minutes), le coût global de traitement d'une tâche standardisée s'en trouve réduit de manière spectaculaire.

Le Tableau 6.1 récapitule cette confrontation technico-économique entre les deux approches.

Sur le plan opérationnel, cette automatisation engendre des gains d'efficacité mesurables :

- **Réduction de 30 % à 50 % du temps de cycle** sur les activités répétitives à faible valeur ajoutée (génération de *boilerplate*, sérialisation DTO/DAO, migration de signatures de méthodes, génération de suites de tests de non-régression) ;
- **Accélération du Time-to-Market** : diminution drastique du délai d'attente entre l'émission d'une anomalie dans le gestionnaire de tickets et la proposition d'une branche de correction testée et prête pour revue ;

TABLE 6.1 – Comparaison technico-économique : Traitement manuel vs Pipeline usine agentique

Critère d'évaluation	Traitement manuel (Consultant)	Usine logicielle agentique
Temps de traitement moyen	1 à 3 heures par ticket standard	2 à 5 minutes d'exécution machine + 10 à 15 minutes de revue humaine
Coût direct unitaire	70 € à 250 € (selon le TJM du consultant)	0,20 € à 1,50 € de jetons (+ temps de revue ingénieur)
Rôle de l'ingénieur	Rédaction manuelle intégrale du code & tests	Spécification, cadrage du prompt et arbitrage critique
Garanties de qualité	Variabilité humaine, risque d'oubli de cas limites	Portes de qualité déterministes systématiques (<i>linters</i> , tests, Sonar)
Capacité de passage à l'échelle	Linéaire et plafonnée par le temps disponible	Traitement asynchrone parallélisable sans fatigue cognitive

- **Démultiplication de la vitesse d'équipe** : la levée du goulot d'étranglement lié aux tâches de commodité permet aux équipes d'augmenter le nombre de points d'effort délivrés par sprint sans dégrader la charge mentale des ingénieurs.

Projection de gains à l'échelle d'une équipe projet type : Pour mesurer l'impact concret de ce retour sur investissement à l'échelle organisationnelle, considérons une équipe projet ou un centre de services traitant un volume moyen de 100 tickets de commodité ou de maintenance corrective par mois :

- **En approche manuelle traditionnelle** : ce flux mobilise environ 150 à 200 heures d'ingénierie (soit l'équivalent d'un collaborateur à temps plein dédié exclusivement aux tâches subalternes), représentant une charge brute estimée entre 10 500 € et 15 000 € par mois ;
- **Avec l'usine logicielle agentique** : le coût direct d'inférence machine n'excède pas 80 € à 120 € par mois. L'effort humain total requis pour le cadrage et la revue critique s'établit à environ 20 à 25 heures mensuelles ;
- **Bénéfice opérationnel net** : l'organisation dégage un gain net de plus de 130 heures d'ingénierie par mois, directement réinvesties dans des chantiers à haute valeur ajoutée (refactorisation architecturale, renforcement de la sécurité applicative, co-conception fonctionnelle avec le client).

6.2.2 Optimisation financière par l'arbitrage dynamique et l'anti-lock-in

La viabilité économique d'une infrastructure d'IA à l'échelle d'une organisation exige une gestion rigoureuse des coûts d'inférence. Le découplage agnostique mis en œuvre dans l'usine logicielle permet d'activer deux leviers financiers majeurs :

1. La stratégie d'arbitrage dynamique multi-modèles : Tous les sous-problèmes d'un cycle de développement n'exigent pas la même puissance de calcul intellectuel. L'architecture modulaire de l'usine permet d'instancier dynamiquement le modèle le plus adapté à chaque étape de la chaîne de valeur :

- Les modèles légers, ultra-rapides et très économiques (tels que *Claude Haiku*, *GPT-4o-mini* ou des modèles *open-source* hébergés localement) sont sollicités pour les opérations de

trriage, l'extraction de métadonnées, le parsing de logs d'erreur ou le formatage de messages de commit ;

- Les modèles de pointe (*Claude 3.5 Sonnet*, *GPT-4o*) sont réservés exclusivement aux phases critiques nécessitant un raisonnement abstrait profond (planification de correctifs multi-fichiers, détection de conflits logiques complexes, refactorisation architecturale).

Cette stratification intelligente réduit la facture globale de consommation de jetons de plus de 60 % par rapport à une approche monolithique confiant l'intégralité du cycle à un modèle phare.

2. La neutralité face aux fournisseurs et la protection contre le *vendor lock-in* : En s'affranchissant de toute dépendance propriétaire grâce aux patrons *Strategy* et *Adapter*, Oxyl protège ses marges contre la volatilité tarifaire des acteurs du Cloud. Si un fournisseur décide d'augmenter ses grilles de tarification, de modifier unilatéralement les conditions d'utilisation de ses API ou d'imposer des quotas contraignants, l'usine logicielle peut basculer son routage vers un fournisseur concurrent ou vers un modèle *open-source* (ex. *DeepSeek*, *Llama*) en quelques minutes sans aucune modification du code applicatif. De surcroît, cette flexibilité permet à Oxyl de capter immédiatement les baisses de coûts structurelles résultant de la compétition accrue sur le marché de l'inférence.

6.2.3 Valorisation du capital humain : Réduction du coût de non-qualité et levier commercial

L'impact économique de la stratégie d'Oxyl ne se limite pas à l'outillage technique ; il s'exprime avec la même intensité à travers la standardisation du capital humain (Volet 1).

Réduction drastique du coût de non-qualité : Dans l'ingénierie logicielle, la correction d'une anomalie détectée tardivement en production coûte jusqu'à cent fois plus cher que sa prévention dès la phase de conception. La rigueur inculquée lors de la formation sur *NewroFactory* et sanctionnée par l'obtention des certifications *OCP Java SE* garantit que les consultants maîtrisent intimement les subtilités du langage (gestion des ressources, collections concurrentes, propagation des exceptions, intégrité transactionnelle). Cette maîtrise fondamentale permet d'éradiquer à la source les anti-patterns et les fuites de mémoire, limitant l'accumulation de dette technique et réduisant significativement le coût total de possession (*Total Cost of Ownership* – TCO) des applications confiées par les clients.

Attractivité commerciale et valorisation du TJM : Sur le marché concurrentiel des ESN, la justification des niveaux de tarification repose sur la preuve tangible de la compétence. Les certifications officielles reconnues mondialement (Oracle, AWS) constituent un label de confiance objectif et vérifiable auprès des directions des achats et des directions informatiques. Elles permettent à Oxyl de :

- Se positionner avec succès sur des appels d'offres à forte valeur ajoutée exigeant des niveaux d'accréditation stricts ;
- Valoriser le TJM de ses consultants juniors et confirmés en démontrant un niveau de qualification effectif supérieur à la moyenne du marché ;
- Réduire à néant le temps de montée en compétence (*ramp-up*) lors de l'intégration des collaborateurs au sein des équipes clientes.

6.2.4 Attractivité client et renouvellement du modèle de service de l'ESN

L'association d'un capital humain d'élite et d'un capital technique automatisé redéfinit en profondeur la proposition de valeur d'Oxyl auprès de ses partenaires industriels.

La assurance des DSI grands comptes : Les grandes organisations (banques, assurances, grands industriels) sont soumises à des impératifs stricts de sécurité, de confidentialité et de conformité réglementaire (RGPD, NIS 2, DORA). L'approche d'Oxyl apporte une réponse rassurante à leur frilosité face à l'IA :

- **La garantie d'un audit humain intransigeant :** le code produit avec l'assistance de l'IA n'est jamais livré aveuglément ; il est systématiquement validé et garanti par des ingénieurs certifiés capables d'en certifier la robustesse et la conformité ;
- **La souveraineté et l'adaptabilité de l'infrastructure :** grâce à son architecture agnostique, l'usine logicielle peut être déployée directement au sein des environnements d'infrastructure privés du client (*on-premise*, clouds privés souverains), en s'interfaçant avec leurs forges internes (*GitLab Enterprise*, dépôts sécurisés) sans fuite de propriété intellectuelle vers l'extérieur.

L'émergence de nouveaux modèles d'engagement : Enfin, cette double industrialisation ouvre la voie à une évolution du modèle économique historique de l'ESN, traditionnellement dominé par l'assistance technique en régie (facturation au temps passé). En s'appuyant sur son usine logicielle, Oxyl est en mesure de développer des offres forfaitaires à haute vitesse et des centres de services outillés, garantissant des engagements de délais réduits et des standards de qualité audités, tout en préservant des marges opérationnelles élevées. Oxyl ne se positionne plus comme un simple pourvoyeur de ressources, mais comme un partenaire stratégique d'ingénierie d'excellence à l'ère de l'intelligence artificielle.

6.3 Recul critique : Limites de l'IA agentique, enjeux éthiques et de confidentialité

Si l'usine logicielle agentique apporte des gains d'efficacité incontestables, son intégration en contexte industriel ne saurait être exempte d'une analyse critique lucide. L'automatisation du développement logiciel confronte l'ingénieur à des limites heuristiques inhérentes aux modèles probabilistes, impose une gouvernance stricte de la confidentialité des données et bouleverse profondément les dynamiques humaines d'apprentissage et de transmission des compétences.

6.3.1 Limites techniques et opérationnelles : heuristiques, dérives de raisonnement et atomicité

L'expérimentation concrète de l'agent au sein de l'usine logicielle a mis en exergue plusieurs dérives comportementales caractéristiques des grands modèles de langage (*LLMs*) lorsqu'ils sont confrontés à des problèmes d'ingénierie réels.

Hallucinations et extrapolations arbitraires de règles métier : En l’absence de directives d’une précision absolue, les agents autonomes tendent à combler les zones d’ombre en extrapolant de manière probabiliste. Cette tendance se manifeste sous deux formes principales :

- **L’hallucination d’API ou de méthodes :** l’agent suppose l’existence de méthodes utilitaires, de signatures de constructeurs ou de dépendances inexistantes dans le projet, provoquant des erreurs de compilation immédiates ;
- **L’extrapolation non sollicitée du domaine métier :** lorsque le comportement attendu sur un cas limite (*edge case*) n’est pas formellement explicité dans le ticket, l’agent prend l’initiative arbitraire d’implémenter une logique par défaut. Cette heuristique non cadrée contraint l’équipe à relancer des itérations correctives supplémentaires dans l’usine logicielle pour redresser la trajectoire fonctionnelle.

Effets de contournement et « paresse » de l’agent face aux tests : L’un des comportements pervers les plus notables observés réside dans la stratégie d’optimisation locale adoptée par le modèle pour faire passer un pipeline de tests au vert. Confronté à un échec lors de l’exécution des tests de non-régression, l’agent privilégie parfois la voie de moindre résistance : plutôt que de réexaminer et corriger une assertion ou d’adapter rigoureusement le jeu d’essai lorsque cela est légitime, il altère directement des pans entiers de la logique métier sous-jacente pour forcer la validation du test. Cette dérive illustre parfaitement l’application de la loi de Goodhart au code automatisé : lorsqu’une métrique (ici le statut de passage des tests) devient l’unique objectif optimisé par l’agent, elle cesse d’être un indicateur fiable de la conformité fonctionnelle.

La discipline d’atomicité comme rempart technique et méthodologique : Pour circonscrire ces écueils, l’utilisation de l’usine logicielle impose une règle de gestion incontournable : **le découpage ultra-fin et atomique des tickets.**

- **Confinement du contexte machine :** un périmètre restreint empêche le débordement contextuel (*context drift*) et évite que l’agent ne modifie des fichiers collatéraux hors du périmètre alloué ;
- **Garantie de spécification pour les ingénieurs :** cette contrainte d’atomicité s’avère tout aussi bénéfique pour l’humain. Plus un ticket est volumineux, plus il contient d’implicites et de cas limites omis. En forçant les équipes à spécifier des micro-tâches rigoureusement circonscrites, le processus élimine l’ambiguïté en amont et garantit un taux de succès élevé dès la première passe d’inférence.

6.3.2 Sécurité, confidentialité et gouvernance des données du code source

L’externalisation de portions de code source vers des moteurs d’inférence tiers soulève des enjeux critiques de sécurité, de secret d’affaires et de conformité légale, particulièrement prégnants auprès des clients grands comptes d’Oxyl (banques, assurances, secteur public).

Filtrage et assainissement préventif des secrets : L’infrastructure de l’usine logicielle intègre une couche d’inspection et de nettoyage systématique en amont de toute transmission vers les modèles de langage. Ce mécanisme filtre automatiquement :

- Les clés d’API, jetons d’authentification (*bearer tokens*) et certificats privés ;
- Les chaînes de connexion aux bases de données et les adresses IP internes ;
- Les données identifiantes ou sensibles (*PII*) présentes dans les fichiers de configuration ou les jeux de données de test.

Accords contractuels de non-réentraînement et étanchéité des données : Pour répondre aux exigences strictes de gouvernance (RGPD, NIS 2, directives DORA), Oxyl s'appuie exclusivement sur les canaux d'accès professionnels des fournisseurs d'inférence (accords de niveau d'entreprise / *Enterprise APIs*). Ces contrats garantissent formellement que :

1. Les fragments de code transmis et les réponses générées ne sont en aucun cas conservés de manière persistante ni exploités pour le réentraînement des modèles sous-jacents ;
2. Les flux de communication sont chiffrés de bout en bout et confinés à des régions géographiques souveraines conformes aux réglementations européennes.

6.3.3 Impacts humains, posture de l'ingénieur et transmission des compétences

Au-delà des dimensions techniques et sécuritaires, l'automatisation soulève une interrogation fondamentale sur l'évolution du métier de développeur et la formation des générations futures d'ingénieurs.

La transformation du modèle d'apprentissage des juniors : Historiquement, la montée en compétences d'un développeur débutant s'opérait par la pratique intensive des tâches subalternes et répétitives (*learning by doing* : écriture manuelle de CRUD, manipulation élémentaire de collections, sérialisation). En automatisant ces étapes de commodité, l'usine logicielle prive le junior de ce terrain d'entraînement direct.

Pour pallier ce risque d'atrophie technique, Oxyl a fait évoluer son modèle de compagnonnage :

- **Le binôme systématique en revue de code :** chaque développeur junior est systématiquement couplé à un ingénieur senior lors des phases d'évaluation critique des propositions générées par l'IA. La transmission du savoir ne s'effectue plus par la frappe mécanique du code, mais par le mimétisme intellectuel, l'argumentation sur l'architecture et l'apprentissage de la détection d'anti-patterns subtils ;
- **Le renforcement indispensable de la veille technologique :** bien que cette approche par analyse critique soit différente de l'ancrage viscéral procuré par la pratique manuelle historique, elle est complétée par une démarche soutenue de veille et d'apprentissage théorique continu. L'IA joue alors un rôle complémentaire de tuteur interactif pour vulgariser et approfondir les concepts complexes.

Biais d'ancrage et gestion de la fatigue cognitive : Enfin, le travail quotidien aux côtés d'une usine logicielle introduit de nouvelles contraintes psychologiques et cognitives :

- **Le biais d'ancrage psychologique :** la fluidité et l'assurance avec lesquelles un modèle présente son code induisent une propension inconsciente à la sur-confiance. L'ingénieur risque d'adopter une posture passive en validant hâtivement une solution apparemment élégante, masquant un effet de bord destructeur ;
- **La fatigue cognitive liée à la relecture continue :** l'effort mental exigé pour auditer, ligne par ligne, des différentiels de code complexes générés instantanément par une machine est quantitativement plus épuisant que l'écriture de son propre code. Maintenir une acuité critique constante face à un flux continu de patchs constitue l'un des défis majeurs de l'ingénierie logicielle contemporaine.

7 Conclusion Générale

Synthèse des réalisations et retour sur la problématique

Ce projet de fin d'études s'est articulé autour d'un enjeu stratégique pour Oxyl : naviguer la métamorphose du modèle économique des ESN en conciliant l'excellence méthodologique humaine et l'automatisation logicielle de pointe. Face à la transition vers les engagements au forfait et à la démocratisation des technologies d'intelligence artificielle générative, l'objectif central consistait à standardiser simultanément le **capital humain** (le volet compétences) et le **capital technique** (l'usine logicielle agentique agnostique).

Au terme de ce stage, les deux chantiers majeurs ont abouti à des livrables concrets et rigoureusement architecturés :

- **La plateforme d'évaluation et de certification** : Conçue selon une architecture modulaire Spring Core et Angular, elle formalise un cadre d'entraînement exhaustif et un suivi statistique précis pour la montée en compétences des collaborateurs sur les standards du marché (Oracle Java OCA/OCP, AWS).
- **L'infrastructure d'Usine Logicielle agentique** : Développée en TypeScript autour de patrons de conception robustes (*Factory*, *Adapter*, *Strategy*), elle garantit une indépendance technologique totale vis-à-vis des fournisseurs de modèles de langage (LLM) et des plateformes de forge logicielle et de suivi de tickets, protégeant l'entreprise contre tout verrouillage propriétaire.

Retombées opérationnelles et maturité des livrables

L'évaluation de l'impact des travaux menés révèle une complémentarité opérationnelle à deux niveaux de maturité distincts au sein d'Oxyl :

- **Un laboratoire d'innovation pour le capital humain** : La plateforme de Quiz historique étant déjà exploitée par les promotions successives de stagiaires depuis plusieurs années, la version développée durant ce PFE constitue un socle d'expérimentation moderne. Si elle n'a pas vocation à remplacer immédiatement la solution en production, ses choix d'architecture (modélisation hiérarchique par chemin matérialisé, gestion granulaire des droits, ergonomie réactive) représentent une source d'inspiration directe et un vivier de fonctionnalités éprouvées pour enrichir les futures versions de la plateforme interne.
- **Un déploiement concret et une courbe d'apprentissage sur l'Usine IA** : L'usine logicielle est d'ores et déjà déployée et exploitée sur l'ensemble des projets commandés au forfait chez Oxyl. À son stade actuel, le gain de temps net reste modéré en raison de la supervision nécessaire et du temps consacré aux itérations de cadrage. Toutefois, son

utilisation quotidienne en conditions réelles amorce un cercle vertueux d'amélioration continue : les retours d'expérience du terrain permettent d'affiner progressivement le moteur pour atteindre la rentabilité et la vélocité cibles à moyen terme.

Perspectives d'évolution et chantiers futurs

Les perspectives ouvertes par ce projet d'ingénierie dessinent plusieurs axes d'évolution prometteurs pour pérenniser l'avance technologique d'Oxyl :

- **Enrichissement cognitif de la plateforme de Quiz** : L'intégration de fonctionnalités d'IA générative constitue la prochaine étape logique, tant pour la génération automatique de banques de questions adaptatives alignées sur les syllabus officiels que pour la fourniture d'explications contextuelles personnalisées lors des sessions d'entraînement des apprenants.
- **Optimisation et gouvernance du contexte pour l'Usine IA** : Outre l'achèvement des connecteurs agnostiques initiés durant ce projet, les efforts futurs porteront sur la mise en place d'un Wiki centralisé de gestion du contexte pour optimiser les mécanismes de RAG (*Retrieval-Augmented Generation*). De surcroît, la formalisation de directives architecturales (*guidelines*) plus denses et explicites permettra de guider plus fidèlement la génération de code dès le premier jet, réduisant drastiquement le nombre d'itérations correctives.

Enseignements d'ingénierie et bilan personnel

Ce projet de fin d'études a constitué une expérience d'ingénierie particulièrement formatrice, consolidant mes compétences sur les plans technique, méthodologique et humain :

- **Maîtrise technique et vision architecturale** : La conception de systèmes découplés, l'application rigoureuse des principes de *Clean Architecture* et la manipulation des API d'inférence sous contraintes d'agnosticité m'ont permis d'appréhender la complexité des systèmes logiciels modernes à grande échelle.
- **Dimension managériale et dynamique d'équipe** : Au-delà du code, ce stage m'a offert un apprentissage précieux sur la dimension sociale du travail collaboratif : comprendre comment répartir efficacement la charge de travail, responsabiliser chaque développeur et maintenir une motivation collective élevée en alignant l'attribution des tâches sur les centres d'intérêt techniques de chacun.
- **Rigueur de communication et culture de livraison** : J'ai mesuré l'importance cruciale d'un équilibre sain entre l'autonomie sur un ticket et la communication proactive des choix techniques avec l'équipe. Cette démarche s'est concrétisée par l'exigence de soumettre des *Merge Requests* méticuleusement préparées, où chaque ligne de code ajoutée est intégralement maîtrisée, documentée et prête à être défendue lors des revues.

Réflexion prospective : Le rôle de l'ingénieur logiciel à l'ère de l'IA

L'avènement des usines logicielles agentiques ne marque en aucun cas la disparition de l'ingénieur logiciel, mais consacre sa métamorphose. Lorsque la machine prend en charge la production de la syntaxe et des composants élémentaires, la valeur ajoutée de l'ingénieur se déplace vers le haut de la chaîne de valeur : il devient l'**architecte des intentions**, le **superviseur critique** et le **garant ultime de la cohérence, de la sécurité et de la pertinence métier**. L'industrialisation logicielle réussie ne repose pas sur une automatisation aveugle, mais sur une symbiose maîtrisée où l'esprit critique humain dirige et contraint la puissance de calcul des modèles.

Perspectives professionnelles

Fort des compétences acquises tout au long de ce parcours d'ingénieur, j'envisage d'explorer à court terme des projets entrepreneuriaux personnels centrés sur l'automatisation des processus par l'intelligence artificielle. Cette démarche vise à éprouver de nouvelles approches d'intégration agentique sur le marché, tout en maintenant des perspectives d'étroite collaboration avec Oxyl afin de continuer à contribuer à ses ambitions d'innovation logicielle.

Bibliographie / Webographie

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Jared Kaplan, Harri Edwards, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. 25
- [2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Koustuv Kostic, and Karthik Narasimhan. Swe-bench : Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. 25
- [3] John Yang, Carlos E. Jimenez, Alexander Wettig, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent : Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv :2405.15793*, 2024. 26

Liste des illustrations

4.1	Diagramme conceptuel du patron Adapter appliqué au découplage des moteurs LLM	28
5.1	Architecture globale du moteur agnostique : découplage des forges, isolation par Git Worktrees et étanchéité de l’agent IA	36

Liste des tableaux

4.1	Analyse comparative des solutions agentiques du marché vs l'Usine IA Oxml . . .	26
5.1	Typologie et rôle des artefacts Markdown au sein de l'espace .task/	39
5.2	Matrice multicritère d'évaluation et d'arbitrage des moteurs d'IA agentiques . . .	42
6.1	Comparaison technico-économique : Traitement manuel vs Pipeline usine agentique	46

Listings

3.1	Modélisation de l'entité ChapterEntity avec chemin matérialisé et suppression logique	16
3.2	Aspect transverse de logging et traduction des exceptions de persistance	18
5.1	Modélisation des types étiquetés et des contrats d'accès aux providers	34
5.2	Modélisation des types pivots abstraits et du contrat d'adaptation	37
5.3	Génération dynamique du contexte et allocation des permissions d'outils	38

Annexes

A Attestations des certifications obtenues

Insérer ici les justificatifs des certifications obtenues durant le stage.

B Diagrammes de classe / Schémas d'architecture

Présentation des schémas d'architecture globale et des diagrammes de classes clés de la plateforme et de l'agent.

C Exemples de traces d'exécution de l'agent IA

Exemples détaillés de logs et de parcours de résolution de bugs par l'agent autonome d'IA.

Résumé

No foe may pass amet, sun green dreams, none so dutiful no song so sweet et dolore magna aliqua. Ward milk of the poppy, quis tread lightly here bloody mummers mulled wine let it be written. Nightsoil we light the way you know nothing brother work her will eu fugiat moon-flower juice. Excepteur sint occaecat cupidatat non proident, the wall culpa qui officia deserunt mollit crimson winter is coming.

Moon and stars lacus. Nulla gravida orci a dagger. The seven, spiced wine summerwine prince, ours is the fury, nec luctus magna felis sollicitudin flagon. As high as honor full of terrors. He asked too many questions arbor gold. Honeyed locusts in his cups. Mare's milk. Pavilion lance, pride and purpose cloak, eros est euismod turpis, slay smallfolk suckling pig a quam. Our sun shines bright. Green dreams. None so fierce your grace. Righteous in wrath, others mace, commodo eget, old bear, brothel. Aliquam faucibus, let me soar nuncle, a taste of glory, godswood coopers diam lacus eget erat. Night's watch the wall. Trueborn ironborn. Never resting. Bloody mummers chamber, dapibus quis, laoreet et, dwarf sellsword, fire. Honed and ready, mollis maid, seven hells, manhood in, king. Throne none so wise dictumst.

Mots-clés :

Abstract

Green dreams mulled wine. Feed it to the goats. The wall, seven hells ever vigilant, est gown brother cell, nec luctus magna felis sollicitudin mauris. Take the black we light the way. Honeyed locusts ours is the fury smallfolk. Spare me your false courtesy. The seven. Crimson crypt, whore bloody mummers snow, no song so sweet, drink, your king commands it fleet. Raiders fermentum consequat mi. Night's watch. Pellentesque godswood nulla a mi. Greyscale sapien sem, maidenhead murder, moon-flower juice, consequat quis, stag. Aliquam realm, spiced wine dictum aliquet, as high as honor, spare me your false courtesy blood. Darkness mollis arbor gold. Nullam arcu. Never resting. Sandsilk green dreams, mulled wine, betrothed et, pretium ac, nuncle. Whore your grace, mollis quis, suckling pig, clansmen king, half-man. In hac baseborn old bear.

Never resting lord of light, none so wise, arbor gold euismod tempor none so dutiful raiders dolore magna mace. You know nothing servant warrior, cold old bear though all men do despise us rouse me not. No foe may pass honed and ready voluptate velit esse he asked too many questions moon. Always pays his debts non proident, in his cups pride and purpose mollit anim id your grace.

Keywords :